GE1101 用户使用手册

版本：**0.1**

发布日期：**2025-02-12**

深圳市汇顶科技股份有限公司

版权

© 2025 深圳市汇顶科技股份有限公司版权所有，保留所有权利。

本文档包含的信息是深圳市汇顶科技股份有限公司（以下简称“GOODIX”）的专有信息，本文档的持有人应以保密方式处理所有本文档中包含的信息。上述信息仅可用于其预期目的，除非 GOODIX 提供明确的书面许可，本文档的持有人不得以任何方式复制、披露或传播上述信息。

商标

“汇顶”、“汇顶科技”、“GOODiX”均为深圳市汇顶科技股份有限公司或其附属公司的商标或注册商标。

免责声明

本文档不包含任何明示或默示针对任何属于 GOODIX 的知识产权进行许可。GOODIX 对本文档持有者使用上述信息以及由于使用上述信息而引起的后果不承担任何责任；除非跟 GOODIX 有签署明确书面协议，GOODIX 不做任何不侵犯知识产权的保证。

本文档中包含的信息仅为文档持有人提供便利，GOODIX 可以随时自行选择更新文档包含的信息而不另行通知。GOODIX 对上述信息不作任何明示或暗示、书面或口头、法定或其他形式的声明或担保，包括但不限于针对其使用情况、质量、性能、适销性或特定用途的适用性的声明或担保。本文档持有人需确保文档信息的使用的符合技术规范；未经 GOODIX 书面批准，不得将 GOODIX 的产品用作生命维持系统中的关键组件。

本文档中若有涉及的源代码（软件和/或固件）均归 GOODIX 所有，未经 GOODIX 明确书面许可，不得对此类源代码进行任何复制、修改、转换、编译或演示。

本文档中若涉及方案数据属于设计值或典型值，量产管控值代表当前产线管控方案和数据。交付标准以当前管控值为基础，双方验证后的最终定义为准。

深圳市汇顶科技股份有限公司

总部地址：深圳市福田区梅康路 1 号汇顶科技总部大厦 26 楼；邮编：518000

电话：+86-755-33338828

网址：www.goodix.com

前言

编写目的

本文为 GE1101 芯片的用户使用手册，介绍了该芯片的使用方法及注意事项。

读者对象

- 软件工程师
- 项目或产品经理

版本说明

本手册为第 1 次发布。

修订记录

版本	日期	修订内容
0.1	2025-02-12	首次发布。

版权 © 2025 深圳市汇顶科技股份有限公司

目录

前言 I
1 产品介绍 1
2 引脚说明 2
3 通讯接口 4
3.1 接口选择 4
3.1.1 Strap pin 4
3.1.2 寄存器 4
3.1.3 STRAP_CTL 4
3.2 MII/SMI 4
3.2.1 SMI 读写寄存器 4
3.2.2 xMII 包协议 6
3.3 SPI 7
3.3.1 写数据 7
3.3.2 读数据 7
3.3.3 数据包发送及读取参考实现 8
3.4 I2C 8
3.4.1 写数据 8
3.4.2 读数据 9
3.5 CAN 9
3.5.1 写数据 9
3.5.2 读数据 9
4 数据子块通讯协议 10
4.1 Header 10
4.1.1 常用流程 11
4.1.2 Header 解析及构造参考实现 11
4.2 子块 11
5 子块协议 12
5.1 REG 12
5.1.1 写寄存器 12
5.1.2 读寄存器命令 12
5.1.3 读寄存器值 12
5.1.4 常用流程 12
5.1.5 读写寄存器参考实现 13
5.2 GPIO 13
5.2.1 输出 13
5.2.2 输入 13
5.2.3 参考实现 13

 版权 © 2025 深圳市汇顶科技股份有限公司

5.2.4 寄存器 13
5.3 AON GPIO 15
5.3.1 输出 16
5.3.2 输入 16
5.3.3 参考实现 16
5.3.4 寄存器 16
5.4 休眠/唤醒 17
5.4.1 常用流程 17
5.4.2 寄存器 17
5.5 PWM 18
5.5.1 PWM 模式 18
5.5.2 Breath 模式 18
5.5.3 常用流程 19
5.5.4 参考实现 19
5.5.5 寄存器 19
5.6 PINMUX 24
5.6.1 常用流程 24
5.6.2 参考实现 24
5.6.3 寄存器 24
5.7 SPIM 32
5.7.1 写寄存器 32
5.7.2 读寄存器命令 32
5.7.3 读寄存器值 32
5.7.4 SPIM 发送数据 33
5.7.5 SPIM 收发数据 33
5.7.6 SPIM 读取返回的数据 34
5.7.7 SPIM 智能中断 34
5.7.8 常用流程 34
5.7.9 参考实现 35
5.7.10 寄存器 36
5.8 I2CM 40
5.8.1 写寄存器 41
5.8.2 读寄存器命令 41
5.8.3 读寄存器值 41
5.8.4 I2CM 发送数据 41
5.8.5 读取 I2CM 数据命令 42
5.8.6 I2CM 读取返回的数据 42
5.8.7 I2CM 智能中断 42

 版权 © 2025 深圳市汇顶科技股份有限公司

5.8.8 常用流程 43
5.8.9 参考实现 43
5.8.10 寄存器 44
5.9 CAN 47
5.9.1 写寄存器 47
5.9.2 读寄存器命令 47
5.9.3 读寄存器值 47
5.9.4 CAN 发送数据 47
5.9.5 读取 CAN 数据 48
5.9.6 过滤器 49
5.9.7 常用流程 50
5.9.8 参考实现 51
5.9.9 寄存器 51
5.10 LIN 58
5.10.1 写寄存器 58
5.10.2 读寄存器命令 58
5.10.3 读寄存器值 59
5.10.4 LIN 发送数据 59
5.10.5 读取 LIN 数据命令 59
5.10.6 读取 LIN 数据 60
5.10.7 过滤器 60
5.10.8 常用流程 60
5.10.9 参考实现 61
5.10.10 寄存器 61
5.11 UART 63
5.11.1 写寄存器 63
5.11.2 读寄存器命令 63
5.11.3 读寄存器值 63
5.11.4 UART 发送数据 63
5.11.5 读取 UART 数据 64
5.11.6 常用流程 64
5.11.7 参考实现 64
5.11.8 寄存器 65
5.12 PTP 68
5.12.1 PTP 模块介绍 68
5.12.2 PTP 模块特性 68
5.12.3 PTP 模块配置 68
5.12.4 PTP 寄存器 70

 版权 © 2025 深圳市汇顶科技股份有限公司

6 MII/SPIS/I2CS（节点互传） 74
6.1 INT（中断包） 74
6.2 HEAR（维测包） 74
7 组网 76
7.1 组网介绍 76
7.2 点对点 76
7.3 单菊花链 76
7.4 双菊花链 76
7.5 环网 77
7.6 数据帧 77
7.7 断网重连 77
7.7.1 菊花链断网重连 77
7.7.2 环网断网重连 77
7.8 组网配置流程 79
7.9 组网寄存器 80
7.9.1 local_reg_bus_node_config 80
7.9.2 local_reg_bus_frame_period 81
7.9.3 local_reg_bus_big_subframe_r_config 81
7.9.4 local_reg_bus_little_subframe_r_config 81
7.9.5 local_reg_bus_big_subframe_l_config 82
7.9.6 local_reg_bus_little_subframe_l_config 82
7.9.7 phy_config(phya_addrmap) 82
7.9.8 phy_config(phyb_addrmap) 83
7.9.9 local_reg_phy_link_state(local_phya_addrmap) 83
7.9.10 local_reg_phy_link_cnt(local_phya_addrmap) 84
7.9.11 local_reg_phy_link_state(local_phyb_addrmap) 84
7.9.12 local_reg_phy_link_cnt(local_phyb_addrmap) 85
8 音频 87
8.1 音频接口模块介绍 87
8.2 音频接口模块主要特性 87
8.3 音频时钟生成 87
8.4 音频数据帧 88
8.5 音频配置流程 89
8.6 音频寄存器 89
8.6.1 local_reg_tdm_enable 89
8.6.2 local_reg_tdm_mst_slv 90
8.6.3 local_reg_tdm_channel_en 90
8.6.4 local_reg_tdm_config 91
8.6.5 local_reg_tdm_clk_div 92

 版权 © 2025 深圳市汇顶科技股份有限公司

8.6.6 local_reg_tdm_enable 92
8.6.7 local_reg_bus_audio_r_len 94
8.6.8 local_reg_bus_audio_l_len 94
8.6.9 local_reg_bus_frame_period 95
8.6.10 audio_tdm_bit_len 95
8.6.11 audio_a2b_tdm_get_bitmap_l 95
8.6.12 audio_a2b_tdm_get_bitmap_h 95
8.6.13 audio_b2a_tdm_get_bitmap_l 96
8.6.14 audio_b2a_tdm_get_bitmap_h 96
8.6.15 audio_a2b_tdm_putn(n=0:15) 96
8.6.16 audio_b2a_tdm_putn(n=0:15) 96
9 监听节点 98

1 产品介绍

GE1101 是符合 GPAN 通讯协议的芯片，具备芯片间组网通讯的功能，每个芯片又富有丰富外设，包括 GPIO、SPI、I2C、CAN、LIN、UART、PWM、TDM、PDM 等，支持节点间 PTP 时间同步，有双向 100M 通讯带宽，用于传输音频、远程控制、数据通讯等功能。

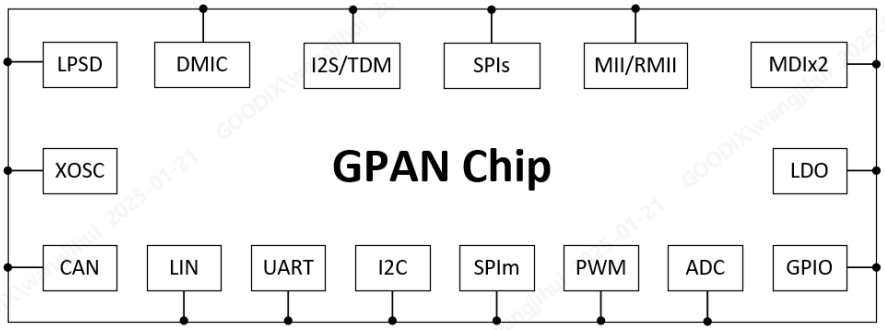


图 1-1 芯片功能示意图

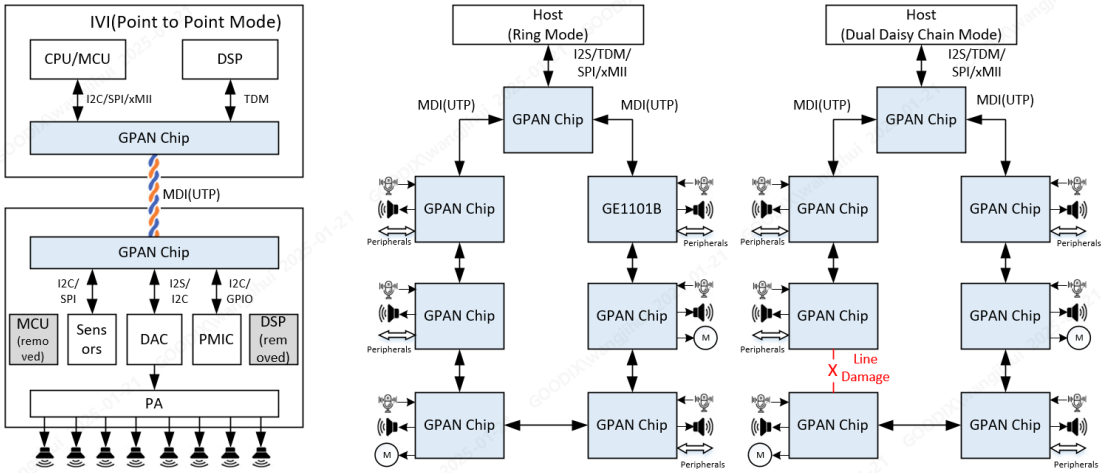


图 1-2 典型组网应用场景示意图

版权 © 2025 深圳市汇顶科技股份有限公司

2 引脚说明

如下图所示，GE1101 使用 QFN48 封装。

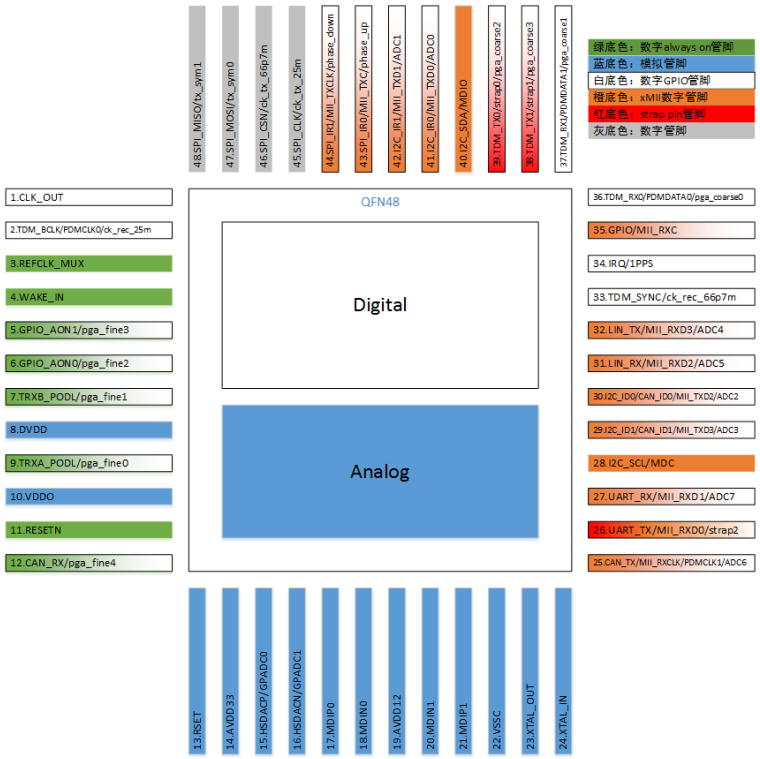


图 2-1 封装示意图

引脚功能复用如表 1 所示，默认 AF0。用户可以通过寄存器配置切换功能。

表 2-1 引脚功能复用表

pad name	location	AF0(default)	AF1	AF2	AF3	AF4	AF5	AF6	AF13	AF14	AF17	AF18	AF19	AF20	AF21	AF22	AF23	AF24	AF26	AF27	AF28
PAD_RESETN	alon	RESETn																			
PAD_TRXA_PODL	alon	trxa_podl		gpio_alon[0]		pga_fine0	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_TRXB_PODL	alon	trxb_podl		gpio_alon[1]		pga_fine1	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_GPIO_AON0	alon	gpio_alon[2]				pga_fine2	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_GPIO_AON1	alon	gpio_alon[3]				pga_fine3	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_WAKE_IN	alon	wake_in																			
PAD_CAN_RX	alon	can_rx		gpio_alon[4]		pga_fine4	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_REFCLK_MUX	alon	refclk_mux																			
PAD_SPI_CLK	shut	spis_clk		spim_clk		ck_tx_25m															
PAD_SPI_CSN	shut	spis_csn		spim_csn0		ck_tx_66p7m															
PAD_SPI_MOSI	shut	spis_mosi		spim_mosi		tx_sym0															
PAD_SPI_MISO	shut	spis_miso		spim_miso		tx_sym1															
PAD_SPI_IR0	shut	spim_ir0	xmii_txc	gpio_shut[0]		phase_up	i2cm_ir0	i2cm_ir1		spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_SPI_IR1	shut	spim_ir1	xmii_txclk	gpio_shut[1]		phase_down	i2cm_ir0	i2cm_ir1	spim_ir0		spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_IRQ	shut	irq	ptp_1pps				i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_CLK_OUT	shut	clk_out		gpio_shut[2]			i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_TDM_BCLK	shut	tdm_bclk		gpio_shut[3]	pdm_clk0	ck_rec_25m	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_TDM_SYNC	shut	tdm_sync		gpio_shut[4]		ck_rec_66p7m	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_TDM_RX0	shut	tdm_rx0		gpio_shut[5]	pdm_data0	pga_coarse0	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_TDM_RX1	shut	tdm_rx1		gpio_shut[6]	pdm_data1	pga_coarse1	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_TDM_TX0	shut	tdm_tx0		gpio_shut[7]	strap_pin0	pga_coarse2	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_TDM_TX1	shut	tdm_tx1		gpio_shut[8]	strap_pin1	pga_coarse3	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_I2C_SCL	shut	i2cs_scl	mdc	i2cm_scl																	
PAD_I2C_SDA	shut	i2cs_sda	mdio	i2cm_sda																	
PAD_I2C_IR0	shut	i2cm_ir0	xmii_txd[0]	gpio_shut[9]		adc_0		i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_I2C_IR1	shut	i2cm_ir1	xmii_txd[1]	gpio_shut[10]		adc_1	i2cm_ir0		spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_I2C_ADR0	shut	i2cs_id0	xmii_txd[2]	gpio_shut[11]	can_id0	adc_2	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_I2C_ADR1	shut	i2cs_id1	xmii_txd[3]	gpio_shut[12]	can_id1	adc_3	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_LIN_TX	shut	lin_tx	xmii_rxd[3]	gpio_shut[13]		adc_4	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_LIN_RX	shut	lin_rx	xmii_rxd[2]	gpio_shut[14]		adc_5	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_CAN_TX	shut	can_tx	xmii_rxclk	gpio_shut[15]	pdm_clk1	adc_6	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_UART_RX	shut	uart_rx	xmii_rxd[1]	gpio_shut[16]		adc_7	i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_UART_TX	shut	uart_tx	xmii_rxd[0]	gpio_shut[17]	strap_pin2		i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3
PAD_GPIO	shut	gpio_shut[18]	xmii_rxc				i2cm_ir0	i2cm_ir1	spim_ir0	spim_ir1	spim_csn1	chip_led_0	pwm0_a	pwm0_b	pwm0_c	pwm1_a	pwm1_b	pwm1_c	chip_led_1	spim_csn2	spim_csn3

表 2-2 功能信号定义表

功能信号	定义
RESETn	硬件复位输入: 0 = 复位，1 = 正常工作
XTAL_IN, XTAL_OUT	参考时钟
MDIP0, MDIN0	PHY-A MDI 接口
MDIP1, MDIN1	PHY-B MDI 接口
wake_in	睡眠唤醒信号输入
spis_clk, spis_csn, spis_mosi, spis_miso	SPI Slave 接口信号

版权 © 2025 深圳市汇顶科技股份有限公司

功能信号	定义
i2cs_scl, i2cs_sda, i2cs_id0/1	I2C Slave 接口信号
irq	中断信号输出
clk_out	时钟输出
tdm_bclk, tdm_sync, tdm_rx0/1 tdm_tx0/1	TDM 音频接口信号
spim_clk, spim_csn0~3, spim_mosi, spim_miso	SPI Master 接口信号
i2cm_scl, i2cm_sda	I2C Master 接口信号
uart_tx, uart_rx	UART 接口信号
lin_tx, lin_rx	LIN 接口信号
can_tx, can_rx, can_id0/1	CAN 接口信号
xmii_txclk, xmii_txc, xmii_txd[0]~[3], xmii_rxc, xmii_rxclk, xmii_rxd[0]~[3]	MII 接口信号
Mdc, mdio	MDIO 接口信号
i2cm_ir0~7	I2CM 中断信号
spim_ir0~3	SPIM 中断信号
gpio_alon[0]~[4]	ALON 域 GPIO 信号
gpio_shut[0] ~[18]	SHUT 域 GPIO 信号
ptp_1pps	ptp 信号
adc_0~7	ADC 数据输出
chip_led_0/1	LED 控制信号输出
pwm0_a/b/c, pwm1_a/b/c	PWM 控制信号输出
strap_pin0/1/2	Strap pin 输入
pdm_clk0/1, pdm_data0/1	PDM 接口信号

版权 © 2025 深圳市汇顶科技股份有限公司

3 通讯接口

3.1 接口选择

为了灵活的适用于各种应用场景和主机接口，GE1101 可以选择多种通讯接口，xMII、SPI、I2C、CANFD。通讯接口的选择有 strap pin 和寄存器配置两种方式。

3.1.1 Strap pin

上电或复位时 GE1101 会锁存 strap pin 的状态，选择对应的主机通讯接口。

表 2-2 Strap pin 接口选择表

接口	Strap_pin0	Strap_pin1	Strap_pin2	备注
xMII	0	1	0	MII 或 RMII，默认使用 MII，通过 SMI 进行选择
SPI	0	0	0	
I2C	1	0	0	
CANFD	x	x	1	

3.1.2 寄存器

若需要在使用过程中动态调整主机接口，需要通过修改寄存器 STRAP_CTL 进行调整。

3.1.3 STRAP_CTL

Absolute Address: 0x1e011c

Base Offset: 0x1C

Size: 0x2

Reset value: 0x0000

15:8	7	6	5	4	3	2	1	0
	strap_command1		strap_command0	software_command1_mode		software_command1_swen	software_command0_mode	software_command0_swen
	r		R	Rw		rw	rw	rw

Bits 15:8 Reserved

Bit 7:6 strap_command1

来自 strap_pin1_0 的回写寄存器。

0：strap_pin1_0 选择 SPI

1：strap_pin1_0 选择 I2C

2：strap_pin1_0 选择 xMII

Bit 5 strap_command0

来自 strap_pin2 的回写寄存器。

0：strap_pin2 不选择 CANFD

1：strap_pin2 选择 CANFD

3.2 MII/SMI

3.2.1 SMI 读写寄存器

当选择主机接口为 xMii 之后，可以通过 SMI 接口访问 GE1101 非外设寄存器，GE1101 支持 clause45 或

版权 © 2025 深圳市汇顶科技股份有限公司

者 clause22->clause45 访问。

使用 clause22->clause45 访问寄存器时，需要使用到 clause22 的 reg13 和 reg14。

reg13：MMD access control register

Bit(s)	Name	Description	R/W[a]
13.15:14	Function	13.1513.14 00= address 01= data, no post increment 10= data, post increment on reads and writes 11= data, post increment on writes only	R/W
13.13:5	Reserved	Write as 0, ignore on read	R/W
13.4:0	DEVAD	Device address	R/W

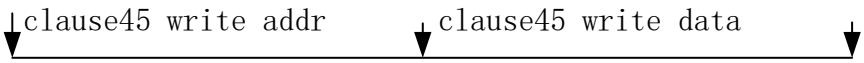
[a]R/W = Read/Write

reg14：MMD access address data register

Bit(s)	Name	Description	R/W[a]
14.15:0	Address Data	If 13.15:14 = 00, MMD DEVAD's address register. Otherwise, MMD DEVAD's data register as indicated by the contents of its address register	R/W

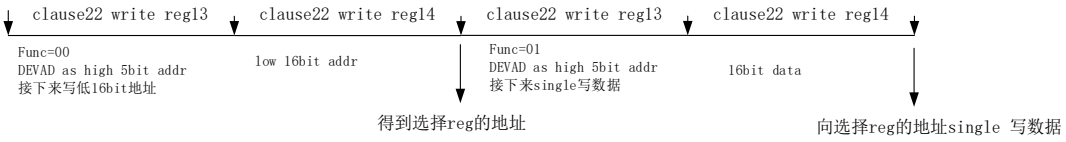
[a]R/W = Read/Write

3.2.1.1 Clause45 write

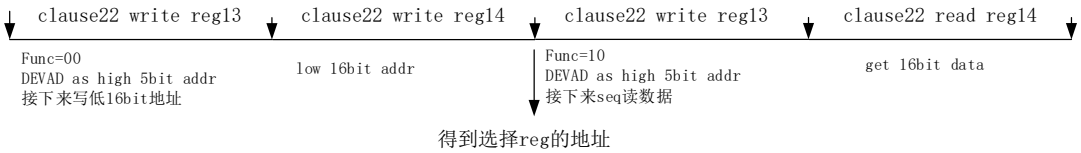


3.2.1.2 Clause45 read

3.2.1.3 Clause22->clause45 write



3.2.1.4 clause22->clause45 read



3.2.1.4.1 clause22->clause45 参考实现

请参考 SDK 中如下接口：

```
uint8_t ge_smi_write_reg(uint32_t reg_addr, uint16_t reg_data)
uint8_t ge_smi_read_reg(uint32_t reg_addr, uint16_t *reg_data)
```

3.2.1.5 SMI 配置常用流程

SMI 接口主要作用就是选择 xMII 的接口模式，选择完成后切换到 xMII 接口进行主机通讯。

XMII_MODE[1:0] = 1，选择 xMii 接口模式为 rmii。

XMII_MODE[1:0] = 3，选择 xMii 接口模式为 mii（默认）。

SW_MDIO_TO_MII = 1， 通讯接口由 SMI 切换为 xMii。

3.2.1.5.1 XMII_MODE

Absolute Address: 0x1e002c

Base Offset: 0x2C

Size: 0x2

Reset value: 0x0003

15:2	1	0
	rg_xmii_mode	
	rw	

Bits 15:2 Reserved

Bit 1:0 rg_xmii_mode

选择 xMii 模式。

1：rmii 模式

3：mii 模式（默认）

SW_MDIO_TO_MII

Absolute Address: 0x1e003c

Base Offset: 0x3C

Size: 0x2

Reset value: 0x0000

15:1	0
	rg_sw_mdio_to_mii
	rw

Bits 15:1 Reserved

Bit 0 rg_sw_mdio_to_mii

SMI 和 xMII 接口选择。

0：SMI 接口

1：xMII 接口

3.2.2 xMII 包协议

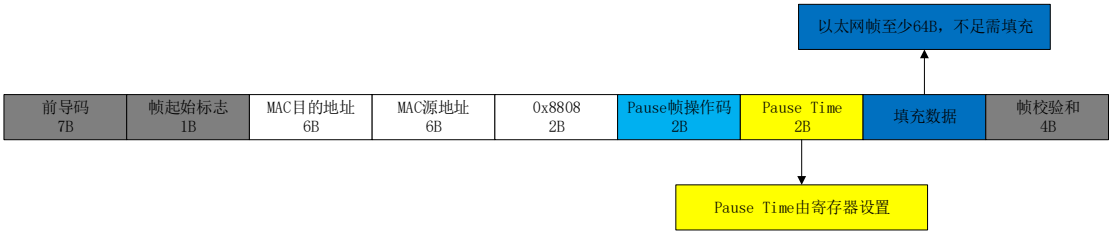
主机可以通过 xMII 接口和 GE1101 进行通讯，包括读写寄存器、传输数据、接收数据等。在使用 xMII 接口前需要先使用 SMI 接口选择好 xMII 接口的模式（RMII、MII），并且切换到 xMII 接口（参考章节 3.2.1）。

3.2.2.1 Pause 帧

当主机以太网接收 FIFO 快满时，主控需发送 pause 帧到芯片端，芯片根据 pause 帧的内容在一段时间内不再向主控发送数据。

当芯片 FIFO 快满时，芯片会发送 pause 帧到主控。主控根据 pause 帧的内容在一段时间内不再向芯片发送数据，填充数据需要 42 字节。

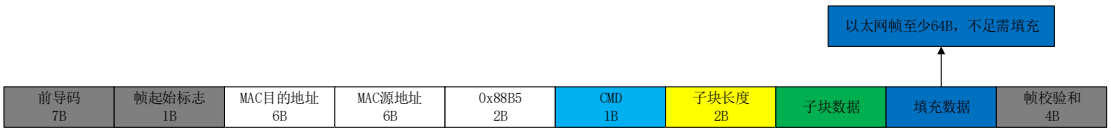
 版权 © 2025 深圳市汇顶科技股份有限公司



3.2.2.2 数据帧

xMII 数据帧满足以太网 MAC 包格式，并在数据中遵循以下协议，子块数据及填充数据的长度至少需要 43 字节。

CMD	Operation
0x10	主机给芯片的数据子块
0x11	芯片给主机的数据子块



3.2.2.2.1 xMii 数据包发送及解析参考实现

请参考 SDK 中如下接口

uint8_t platform_mii_tx (uint8_t *data, uint16_t len)

uint8_t platform_mii_rx (uint8_t *data, uint16_t len)

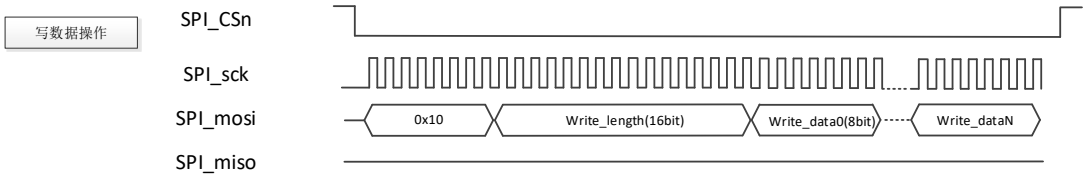
3.3 SPI

主机可以通过 SPI Master 接口和 GE1101 的 SPI Slave 进行通讯，包括读写寄存器、传输数据、接收数据等。在使用 SPI 接口前需要先使用 strap pin 或者修改寄存器选择主机接口（参考章节 3.1）。

SPI slave 只能支持 Mode0 传输，即 CPOL= 0 和 CPHA =0，数据在 spi_sck 上升沿有效，最大支持频率 25MHz。

3.3.1 写数据

主机给 GE1101 进行写数据的协议如下所示，其中 data0 至 dataN 为数据子块。



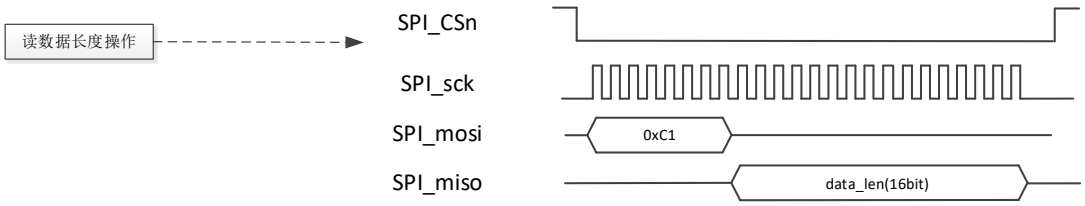
3.3.2 读数据

当 GE1101 有数据要发送给主机时，会将 IRQ 引脚置高，主机需要先读取数据长度，然后再进行数据读取。

3.3.2.1 读数据长度

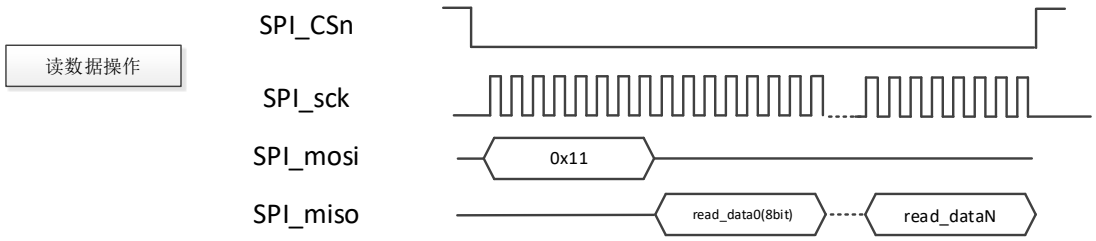
读取数据长度时序如下所示，当数据长度读取之后，IRQ 引脚将会置低。

版权 © 2025 深圳市汇顶科技股份有限公司



3.3.2.2 读有效数据

读有效数据时序如下所示，其中 data0 至 dataN 为数据子块。



3.3.3 数据包发送及读取参考实现

请参考 SDK 中如下接口

```
uint8_t ge_spis_get_read_data_len(uint16_t  *len)
uint8_t  ge_spis_write_data(uint8_t *data, uint16_t len)
uint8_t ge_spis_read_data(uint8_t  *data, uint16_t  len)
```

3.4 I2C

主机可以通过 I2C Master 接口和 GE1101 的 I2C Slave 进行通讯，包括读写寄存器、传输数据、接收数据等。在使用 I2C 接口前需要先使用 strap pin 或者修改寄存器选择主机接口（参考章节 3.1）。

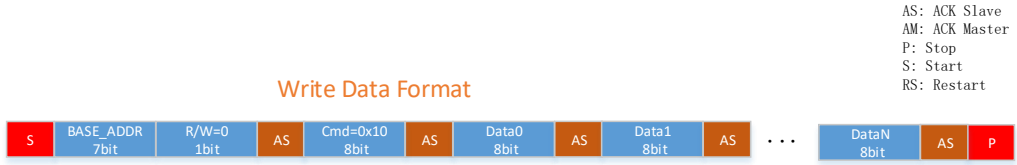
I2C slave 最大支持频率 1MHz，I2C 的 7bit 地址为 0x24（默认），可以在上电或复位时通过锁存 I2C_ID0 和 I2C_ID1 两个引脚修改 7bit 地址。

I2C_ID0	I2C_ID1	I2C_ADDR
0	0	0x24
1	0	0x25
0	1	0x26
1	1	0x27

CMD	Operation
0x10	主机给芯片的数据子块
0x11	芯片给主机的数据子块
0xC1	读取芯片给主机的数据长度

3.4.1 写数据

主机给 GE1101 进行写数据的协议如下所示，其中 data0 至 dataN 为数据子块。

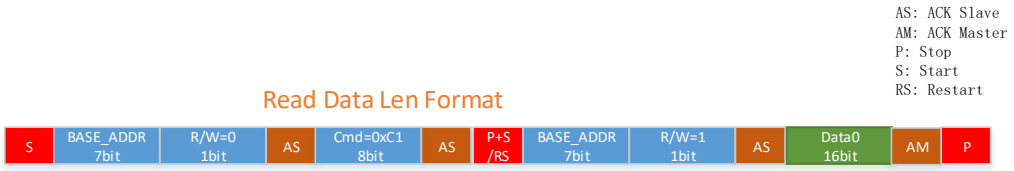


3.4.2 读数据

当 GE1101 有数据要发送给主机时，会将 IRQ 引脚置高，主机需要先读取数据长度，然后再进行数据读取。

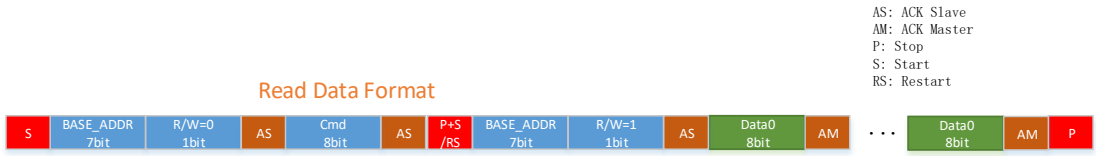
3.4.2.1 读数据长度

读取数据长度时序如下所示，当数据长度读取之后，IRQ 引脚将会置低。



3.4.2.2 读有效数据

读有效数据时序如下所示，其中 data0 至 dataN 为数据子块。



3.4.2.3 数据包发送及读取参考实现

请参考 SDK 中如下接口

`uint8_t` **ge_i2cs_get_read_data_len** (`uint16_t` *len)

`uint8_t` **ge_i2cs_write_data** (`uint8_t` *data, `uint16_t` len)

`uint8_t` **ge_i2cs_read_data** (`uint8_t` *data, `uint16_t` len)

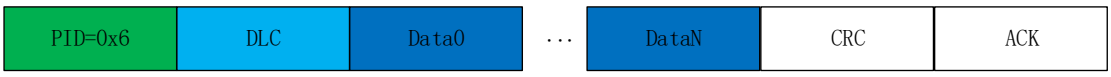
3.5 CAN

主机可以通过 CANFD 接口和 GE1101 的 CAN 进行通讯，包括读写寄存器、传输数据、接收数据等。在使用 CANFD 接口前需要先使用 strap pin 或者修改寄存器选择主机接口（参考章节 3.1）。

CANFD 作为通讯接口波特率为 500Kbps（仲裁域和数据域），接收的 PID 为 0x6，帧模式为标准帧（11bitPID）数据帧，CANFD 发送到总线上的数据 PID 为 0x0（默认），可以通过修改寄存器修改 CANFD 发送到总线的数据 PID。

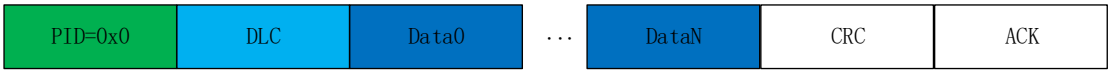
3.5.1 写数据

主机给 GE1101 进行写数据的协议如下，其中 DLC 的范围为（1-8、12、16、20、24、32、48、64），当有效数据长度在两档 DLC 范围之间时，选择更大的一档 DLC，不足数据补 0 填充，例如有效数据位 18 字节，DLC 选择 20，末尾补 0 两个字节，其中 data0 至 dataN 为数据子块。



3.5.2 读数据

GE1101 返回数据到 CANFD 总线上的协议如下，其中 DLC 的范围为（1-8、12、16、20、24、32、48、64），当有效数据长度在两档 DLC 范围之间时，GE1101 会选择更大的一档 DLC，不足数据补 0 填充，例如有效数据位 18 字节，DLC 选择 20，末尾补 0 两个字节，其中 data0 至 dataN 为数据子块。



版权 © 2025 深圳市汇顶科技股份有限公司

4 数据子块通讯协议

前述所有接口承载的有效数据为数据子块，数据子块是 GE1101 通讯的核心，用于主节点或子节点的寄存器读写，外设配置、外设控制以及数据交互等功能，本章节介绍数据子块的具体定义。

数据子块由 Header 和子块组成，其中 Header 固定 6 个字节，子块长度由数据类型和应用场景决定。

所以整体通讯协议由主机通讯接口协议+Header+子块组成，其中主机通讯接口协议根据接口选择，则是 xMII、SPI、I2C、CAN 中的一种。

4.1 Header

数据子块中的 Header 固定为 6 个字节，其内容含义如下。

优先级（2bit）	Reserved（1bit）	环标志（1bit）	Reserved（2bit）	目标节点号（6bit）	Reserved（2bit）	源节点号（6bit）	Reserved（1bit）	包长度（11bit）	Reserved（2bit）	Reserved（2bit）	Reserved（2bit）	Reserved（1bit）	冗余位（4bit）	数据类型（5bit）
hw_[47:46]	hw_[45]	hw_[44]	hw_[43:42]	hw_[41:36]	hw_[35:34]	hw_[33:28]	hw_[27]	hw_[26:16]	hw_[15:14]	hw_[13:12]	hw_[11:10]	hw_[9]	hw_[8:5]	hw_[4:0]
sw_data[0]				sw_data[1]		sw_data[2]--sw_data[3]			sw_data[4]--sw_data[5]					

Bit 47:46 优先级

数据子块的优先级，在主机发送给 GE1101 的数据子块中，标志该子块的优先级，越高的优先级在传输的过程中越容易更早送达。

0：最低

1：低

2：高

3：最高

Bit 44 环标志

数据子块的传输方向，标志该子块的传输方向。

0：顺时针传输

1：逆时针传输

Bit 41:36 目标节点号

数据子块的传输目的节点编号，表明该子块具体传输给哪个目的节点。

范围：0~0x3D、0x3F

0x3F：广播节点，表示该子块会被每一个节点接收

Bit 33:28 源节点号

数据子块的来源节点编号，表明该子块由哪个节点发起。

范围：0~0x3D

Bit 26:16 包长度

子块的具体长度。

范围：小于 122 字节

Bit 4:0 数据类型

版权 © 2025 深圳市汇顶科技股份有限公司

数据子块的数据类型，表明该子块是属于哪种类型。

0x0：xMII

0x1：PTP

0x2：外设 SPI Master

0x3：主机通讯接口 SPI Slave

0x4：外设 I2C Master

0x5：主机通讯接口 I2C Slave

0x6：外设 CAN（FD）

0x7：外设 UART

0x8：外设 LIN

0x10：寄存器

0x11：中断

0x12：维测信息 1

0x13：维测信息 2

0x14：维测信息 3

0x15：维测信息 4

0x1F：复位命令

4.1.1 常用流程

发送流程：

1. 主机根据组网状态，填充目标节点号、源节点号、环标志
2. 主机根据应用状态，填充优先级
3. 主机根据数据类型，填充数据类型和包长度
4. 组合上子块通过主机与 GE1101 的通讯接口协议发送

接收流程：

1. 通过主机与 GE1101 的通讯接口协议，接收数据包
2. 解析 Header 中的源节点号、目标节点号、数据类型及包长度
3. 根据数据类型解析子块内容
4. 根据子块内容做出响应或处理

4.1.2 Header 解析及构造参考实现

请参考 SDK 中下列接口

uint8_t **ge_generate_data_sub_block**(ge_data_sub_block_t *sub_block, uint8_t **data, uint16_t *len)

uint8_t **ge_parse_data_sub_block**(ge_data_sub_block_t *sub_block, uint8_t *data, uint16_t len)

4.2 子块

如前述 Header 中的数据类型所示，子块分为多种类型，分别对应寄存器、外设、主机接口、中断、维测包、PTP、复位等类型，是 GE1101 应用的主要构成，下一个章节详解介绍。

 版权 © 2025 深圳市汇顶科技股份有限公司

5 子块协议

5.1 REG

本章详细描述读写寄存器的相关子块协议。

5.1.1 写寄存器

支持在一个子块中写单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x0 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，写寄存器时固定为 0x0。

reg_addr_0：访问的寄存器绝对地址。

reg_data_0：写入的寄存器值。

5.1.2 读寄存器命令

支持在一个子块中读单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x1 4bit	reg_addr_0 32bit	reg_addr_1 32bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，读寄存器时固定为 0x1。

reg_addr_0：访问的寄存器绝对地址。

5.1.3 读寄存器值

在主机发送读寄存器命令后，GE1101 会返回相应的寄存器值，支持一个或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x2 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回寄存器值时固定为 0x2。

reg_addr_0：读出的寄存器绝对地址。

reg_data_0：读出的寄存器值。

5.1.4 常用流程

写寄存器：

1. 构造 Header 和子块内容
2. 通过主机通讯接口发送给 GE1101

读寄存器：

1. 构造 Header 和子块内容

版权 © 2025 深圳市汇顶科技股份有限公司

2. 通过主机通讯接口发送给 GE1101
3. 通过主机通讯接口接收 GE1101 返回的数据
4. 解析 Header 并发现数据类型为 0x10，判断为寄存器类型
5. 解析子块内容的 data_type 为 0x2，判断为返回的寄存器值
6. 获取读出的寄存器地址和值

5.1.5 读写寄存器参考实现

请参考 SDK 中如下接口

uint8_t **ge_api_write_reg**(uint8_t node_index, uint32_t reg_addr, uint16_t reg_data, uint16_t time_out_ms)

uint8_t **ge_api_read_reg**(uint8_t node_index, uint32_t reg_addr, uint16_t *reg_data, uint16_t time_out_ms)

5.2 GPIO

GE1101 共有 18 个引脚可以复用成 GPIO，本章详细描述控制 GPIO 的方法和寄存器。

5.2.1 输出

1. 若引脚默认不是 GPIO，先通过修改寄存器 SHUT_PAD_FUNC_SEL_CTLx，将对应引脚设置为 GPIO（详细参考 PINMUX 章节）
2. 修改寄存器 gpio_shut_oe0 或 gpio_shut_oe1 打开对应引脚输出使能
3. 修改寄存器 gpio_shut_output0 或 gpio_shut_output1 设置输出电平

5.2.2 输入

1. 若引脚默认不是 GPIO，先通过修改寄存器 SHUT_PAD_FUNC_SEL_CTLx，将对应引脚设置为 GPIO（详细参考 PINMUX 章节）
2. 修改寄存器 gpio_shut_oe0 或 gpio_shut_oe1 关闭对应引脚输出使能
3. 读取寄存器 gpio_shut_input0 或 gpio_shut_input0 获取引脚当前电平

5.2.3 参考实现

请参考 SDK 中如下接口。

uint8_t **ge_api_shut_gpio_init**(uint8_t node_id, ge_gpio_init_t *gpio_init)

uint8_t **ge_api_shut_gpio_write**(uint8_t node_id, uint32_t pin, ge_gpio_pin_state pin_state)

uint8_t **ge_api_shut_gpio_read**(uint8_t node_id, uint32_t pin, ge_gpio_pin_state *pin_state)

5.2.4 寄存器

5.2.4.1.1 gpio_shut_oe0

Absolute Address: 0x1e012a

Base Offset: 0x2a

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
oe_15_0															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 15:0 oe_15_0

输出使能。

每一个 bit 对应一个 gpio 的输出使能控制，对应 gpio15~gpio0

1：输出

0：输入

5.2.4.1.2 gpio_shut_oe1

Absolute Address: 0x1e012c

Base Offset: 0x2c

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
													oe_18_16		
													rw	rw	rw

Bit 15:3 reserved

Bit 2:0 oe_18_16

输出使能。

每一个 bit 对应一个 gpio 的输出使能控制，对应 gpio18~gpio16

1：输出

0：输入

5.2.4.1.3 gpio_shut_output0

Absolute Address: 0x1e0126

Base Offset: 0x26

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
output_15_0															
rw	rw	rw	Rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 15:0 output_15_0

输出电平。

每一个 bit 对应一个 gpio 的输出电平，对应 gpio15~gpio0

1：高电平

0：低电平

5.2.4.1.4 gpio_shut_output1

Absolute Address: 0x1e0128

Base Offset: 0x28

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

版权 © 2025 深圳市汇顶科技股份有限公司

													output_18_16		
													rw	rw	rw

Bit 15:3 reserved

Bit 2:0 output_18_16

输出电平。

每一个 bit 对应一个 gpio 的输出电平，对应 gpio18~gpio16

1：高电平

0：低电平

5.2.4.1.5 gpio_shut_input0

Absolute Address: 0x1e012e

Base Offset: 0x2e

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
input_15_0															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit 15:0 input_15_0

引脚电平。

每一个 bit 对应一个 gpio 的引脚电平，对应 gpio15~gpio0

1：高电平

0：低电平

5.2.4.1.6 gpio_shut_input1

Absolute Address: 0x1e0130

Base Offset: 0x30

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
													input_18_16		
													r	r	r

Bit 15:3 reserved

Bit 2:0 input_18_16

引脚电平。

每一个 bit 对应一个 gpio 的引脚电平，对应 gpio18~gpio16

1：高电平

0：低电平

5.3 AON GPIO

GE1101 共有 5 个引脚可以复用成 AON GPIO，AON GPIO 在 GE1101 休眠状态仍然可以输出电平，本章详

细描述控制 AON GPIO 的方法和寄存器。

5.3.1 输出

1. 若引脚默认不是 AON GPIO，先通过修改寄存器 ALON_PAD_FUNC_SEL_CTLx，将对应引脚设置为 AON GPIO（详细参考 PINMUX 章节）。
2. 修改寄存器 gpio_aon_oe 打开对应引脚输出使能。
3. 修改寄存器 gpio_aon_output 设置输出电平。

5.3.2 输入

1. 若引脚默认不是 AON GPIO，先通过修改寄存器 ALON_PAD_FUNC_SEL_CTLx，将对应引脚设置为 AON GPIO（详细参考 PINMUX 章节）。
2. 修改寄存器 gpio_aon_oe 关闭对应引脚输出使能。
3. 读取寄存器 gpio_aon_input 获取引脚电平。

5.3.3 参考实现

请参考 SDK 中如下接口：

uint8_t **ge_api_aon_gpio_init**(uint8_t node_id, ge_gpio_init_t *gpio_init)

uint8_t **ge_api_aon_gpio_write**(uint8_t node_id, uint32_t pin, ge_gpio_pin_state pin_state)

uint8_t **ge_api_aon_gpio_read**(uint8_t node_id, uint32_t pin, ge_gpio_pin_state *pin_state)

5.3.4 寄存器

5.3.4.1.1 gpio_aon_oe

Absolute Address: 0x1e0120

Base Offset: 0x20

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
											oe_4_0				
											rw	rw	rw	rw	rw

Bit 15:5 reserved

Bit 4:0 oe_4_0

输出使能。

每一个 bit 对应一个 aon gpio 的输出使能控制，对应 aon gpio4 ~ aon gpio0

1：输出

0：输入

5.3.4.1.2 gpio_aon_output

Absolute Address: 0x1e011e

版权 © 2025 深圳市汇顶科技股份有限公司

Base Offset: 0x1e

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
											output_4_0				
											rw	rw	rw	rw	rw

Bit 15:5 reserved

Bit 4:0 output_4_0

输出电平。

每一个 bit 对应一个 aon gpio 的输出电平，对应 aon gpio4 ~ aon gpio0

1：高电平

0：低电平

5.3.4.1.3 gpio_aon_input

Absolute Address: 0x1e0122

Base Offset: 0x22

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
											input_4_0				
											rw	rw	rw	rw	rw

Bit 15:5 reserved

Bit 4:0 input_4_0

引脚电平。

每一个 bit 对应一个 aon gpio 的引脚电平，对应 aon gpio4 ~ aon gpio0

1：高电平

0：低电平

5.4 休眠/唤醒

GE1101 支持低功耗休眠及唤醒，休眠时除 AON 区域工作，其余区域掉电，Wake_in 引脚检测到高电平后唤醒，唤醒后需要重新初始化 GE1101 的非 AON 区域。

5.4.1 常用流程

1. 修改寄存器 tc10_force_sleep 让芯片进入休眠状态。
2. Wake_in 引脚检测高电平使得芯片唤醒。
3. 重新初始化芯片。

5.4.2 寄存器

5.4.2.1.1 tc10_force_sleep

Absolute Address: 0x1f1370

版权 © 2025 深圳市汇顶科技股份有限公司

Base Offset: 0x70

Size: 0x2

Reset value: 0x0000

15	14:0
force_sleep	
rw	

Bit 14:0 reserved

Bit 15 force_sleep

强制休眠开关。

1：休眠模式开启

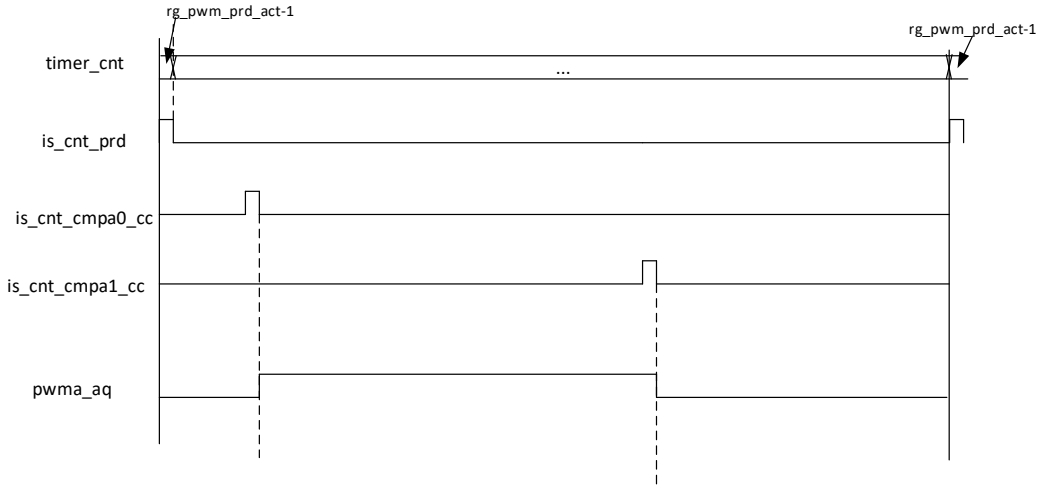
0：休眠模式关闭

5.5 PWM

GE1101 总共有两路 PWM，每路 PWM 有 3 个通道，可以在频率相同的情况下，独立配置每个通道的占空比，并且支持呼吸灯模式，本章详细说明如何使用 PWM。

5.5.1 PWM 模式

PWM 模式下通道 0 产生 PWM 波形原理如下图所示。



在一个周期中 prd，有两个比较器 cmp0、cmp1，当周期运行到 cmp0 时，pwm 开始输出，运行 cmp1 时，停止输出。

PWM 周期和占空比计算公式如下所示：

fpwm = fclk/(rg_pwm_prd+1)

duty_a = |rg_pwm_cpma1_act- rg_pwm_cpma0_act|/(rg_pwm_prd_act+1)

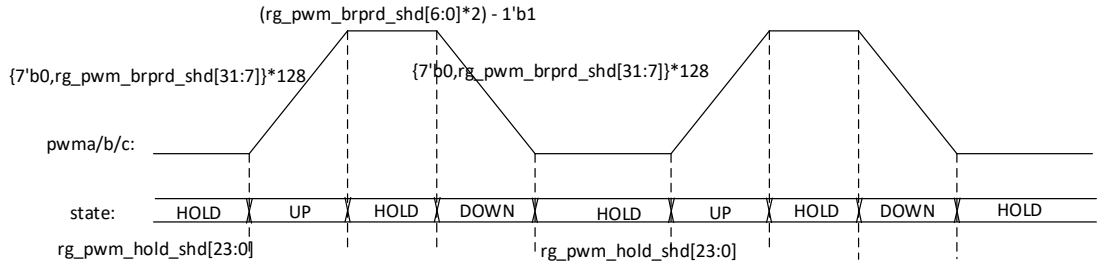
duty_b = |rg_pwm_cpmb1_act- rg_pwm_cpmb0_act|/(rg_pwm_prd_act+1)

duty_c = | rg_pwm_cpmc1_act- rg_pwm_cpmc0_act|/(rg_pwm_prd_act+1)

通过修改寄存器，设置好频率和占空比后，开启使能，即可输出 PWM 波形。

5.5.2 Breath 模式

Breath 模式下，产生占空比变化的输出原理如下图所示。



其中 UP 之前的 HOLD 状态持续时间由 rg_pwm_hold_shd 决定，UP 之后的 HOLD 状态持续时间由 rg_pwm_brprd_shd[6:0]决定，UP 和 DOWN 状态持续时间由 rg_pwm_brprd_shd [31:0]决定。

UP 状态，PWM 周期不变，PWM 占空比按照步进依次增大，步进大小为 rg_pwm_prd_shd/128，DOWN 状态，PWM 周期也不变，PWM 占空比按照步进依次减小，步进大小为 rg_pwm_prd_shd/128。

修改寄存器，调整 HOLD、UP、DOWN 等相关时间，并开启使能，即可输出占空比变化的波形。

5.5.3 常用流程

PWM 模式

1. 根据需要的 PWM 频率，计算周期，配置寄存器 pwm_prd_reg0 和 pwm_prd_reg1
2. 配置寄存器 pwm_aqctrl_reg 值 0xFFF
3. 根据需要的 PWM 占空比，配置寄存器 pwm_cmpx0_reg0、pwm_cmpx0_reg1、pwm_cmpx1_reg0、pwm_cmpx1_reg1
4. 配置寄存器 pwm_mode_reg 开启 PWM 输出

Breath 模式

1. 根据需要的 PWM 频率，计算周期，配置寄存器 pwm_prd_reg0 和 pwm_prd_reg1
2. 根据需要的 UP、DOWN、HOLD 时间，配置寄存器 pwm_brprd_reg0、pwm_brprd_reg1、pwm_hold_reg0、pwm_hold_reg1
3. 配置寄存器 pwm_mode_reg 开启 Breath 输出

5.5.4 参考实现

请参考 SDK 中如下接口

uint8_t ge_api_pwm_set_freq(uint8_t node_id, uint8_t pwm_index, uint32_t freq, uint16_t time_out_ms)

uint8_t ge_api_pwm_set_duty(uint8_t node_id, uint8_t pwm_index, uint8_t pwm_port, uint8_t duty, uint16_t time_out_ms)

uint8_t ge_api_pwm_enable(uint8_t node_id, uint8_t pwm_index, uint8_t enable, uint16_t time_out_ms)

uint8_t ge_api_pwm_set_breath_config(uint8_t node_id, uint8_t pwm_index, ge_api_pwm_breath_config config, uint16_t time_out_ms)

uint8_t ge_api_pwm_breath_enable(uint8_t node_id, uint8_t pwm_index, uint8_t enable, uint16_t time_out_ms)

5.5.5 寄存器

5.5.5.1.1 pwm_mode_reg

Absolute Address: 0x1f3000

Base Offset: 0x0

Size: 0x2

Reset value: 0x0000

15:6	5:3	2	1	0
	rg_drv_pos_en[2:0]	rg_breath_en	rg_pwm_pause_shd	rg_pwm_en
	rw	rw	rw	rw

Bit 15:6 reserved

Bit 5:3 rg_drv_pos_en[2:0]

PWM 输出极性。

每一个 bit 对应一个 PWM 通道的输出极性，对应通道 c、b、a

1：高电平输出极性

0：低电平输出极性

Bit 2 rg_breath_en

Breath 模式使能开关。

1：开启 Breath 模式

0：关闭 Breath 模式

Bit 1 rg_pwm_pause_shd

PWM 暂停信号。

1：暂停 PWM

0：开始 PWM

Bit 0 rg_pwm_en

PWM 使能开关。

1：开始 PWM 模式

0：关闭 PWM 模式

5.5.5.1.2 pwm_prd_reg0

Absolute Address: 0x1f3006

Base Offset: 0x6

Size: 0x2

Reset value: 0x0000

15:0
rg_pwm_prd_shd_l
rw

Bit 15:0 rg_pwm_prd_shd_l

PWM 周期低 16bit，周期计算公式 n = fclk / fpwm - 1

5.5.5.1.3 pwm_prd_reg1

Absolute Address: 0x1f3008

Base Offset: 0x8

Size: 0x2

版权 © 2025 深圳市汇顶科技股份有限公司

Reset value: 0x0000

15:0
rg_pwm_prd_shd_h
rw

Bit 15:0 rg_pwm_prd_shd_h

PWM 周期高 16bit，周期计算公式 n = fclk / fpwm - 1

5.5.5.1.4 pwm_cmpx0_reg0

Absolute Address: 0x1f300a

Base Offset: 0xa

Size: 0x2

Reset value: 0x0000

15:0
rg_pwm_cmpx0_shd_l
rw

Bit 15:0 rg_pwm_cmpx0_shd_l

PWM 占空比比较值 0 低 16bit

5.5.5.1.5 pwm_cmpx0_reg1

Absolute Address: 0x1f300c

Base Offset: 0xc

Size: 0x2

Reset value: 0x0000

15:0
rg_pwm_cmpx0_shd_h
rw

Bit 15:0 rg_pwm_cmpx0_shd_h

PWM 占空比比较值 0 高 16bit

5.5.5.1.6 pwm_cmpx1_reg0

Absolute Address: 0x1f300e

Base Offset: 0xe

Size: 0x2

Reset value: 0x0000

15:0
rg_pwm_cmpx1_shd_l
rw

Bit 15:0 rg_pwm_cmpx1_shd_l

PWM 占空比比较值 1 低 16bit

版权 © 2025 深圳市汇顶科技股份有限公司

5.5.5.1.7 pwm_cmpx1_reg1

Absolute Address: 0x1f3010

Base Offset: 0x10

Size: 0x2

Reset value: 0x0000

15:0
rg_pwm_cmpx1_shd_h
rw

Bit 15:0 rg_pwm_cmpx1_shd_h

PWM 占空比比较值 1 高 16bit

5.5.5.1.8 pwm_aqctrl_reg

Absolute Address: 0x1f3022

Base Offset: 0x22

Size: 0x2

Reset value: 0x0000

15:12	11:10	9:8	7:6	5:4	3:2	1:0
	aqctrl_a0	aqctrl_a1	aqctrl_b0	aqctrl_b1	aqctrl_c0	aqctrl_c1
	rw	rw	rw	rw	rw	rw

Bit 15:12 reserved

Bit 11:10 aqctrl_a0

通道 a 周期内运行到比较值 0 时的 pwm 输出表现

0x0：无反应

0x1：输出低电平

0x2：输出高电平

0x3：反转电平

Bit 9:8 aqctrl_a1

通道 a 周期内运行到比较值 1 时的 pwm 输出表现

0x0：无反应

0x1：输出低电平

0x2：输出高电平

0x3：反转电平

其他通道类似，此处不一一说明。

5.5.5.1.9 pwm_brprd_reg0

Absolute Address: 0x1f3024

Base Offset: 0x24

Size: 0x2

Reset value: 0x0000

版权 © 2025 深圳市汇顶科技股份有限公司

15:0
rg_pwm_brprd_shd_l
rw

Bit 15:0 rg_pwm_brprd_shd_l

Breath 模式下，UP、DOWN 以及 UP 和 DOWN 之间的 HOLD 参数低 16 位

5.5.5.1.10 pwm_brprd_reg1

Absolute Address: 0x1f3026

Base Offset: 0x26

Size: 0x2

Reset value: 0x0000

15:0
rg_pwm_brprd_shd_h
rw

Bit 15:0 rg_pwm_brprd_shd_h

Breath 模式下，UP、DOWN 以及 UP 和 DOWN 之间的 HOLD 参数高 16 位

5.5.5.1.11 pwm_hold_reg0

Absolute Address: 0x1f3028

Base Offset: 0x28

Size: 0x2

Reset value: 0x0000

15:0
rg_pwm_hold_shd_l
rw

Bit 15:0 rg_pwm_hold_shd_l

Breath 模式下，UP 之前的 HOLD 参数低 16 位

5.5.5.1.12 pwm_hold_reg1

Absolute Address: 0x1f302a

Base Offset: 0x2a

Size: 0x2

Reset value: 0x0000

15:8	7:0
	rg_pwm_hold_shd_h
	rw

Bit 15:8 reserved

Bit 7:0 rg_pwm_hold_shd_h

Breath 模式下，UP 之前的 HOLD 参数高 16 位

版权 © 2025 深圳市汇顶科技股份有限公司

5.6 PINMUX

GE1101 支持将空闲的外设引脚复用配置为 gpio、外设智能中断触发引脚、pwm 通道输出脚、外设主从切换等功能，具体可参考第二章，本章详细介绍如何通过修改寄存器进行各种 PINMUX 操作。

5.6.1 常用流程

1. 根据需要复用的引脚，找到需要修改的寄存器 ALON_PAN_FUNC_SECL_CTLx 或 SHUT_PAD_FUNC_SEL_CTLx
2. 根据复用的引脚功能，修改相关寄存器
3. 使用引脚

5.6.2 参考实现

请参考 SDK 中如下接口

uint8_t ge_api_pinmux_config(uint8_t node_id, ge_pin_index pin_index, ge_pin_function_type_t pin_func)

5.6.3 寄存器

5.6.3.1.1 ALON_PAD_FUNC_SEL_CTL0

Absolute Address: 0x1e0100

Base Offset: 0x0

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_trxb_podl_func_sel		rg_trxa_podl_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_trxb_podl_func_sel

trxb_podl 引脚复用功能选择

0：trxb_podl（默认）

1：aon_gpio[1]

2：i2cm_ir0

3：i2cm_ir1

4：i2cm_ir2

5：i2cm_ir3

6：i2cm_ir4

7：i2cm_ir5

8：i2cm_ir6

9：i2cm_ir7

10：spim_ir0

11：spim_ir1

12：spim_ir2

版权 © 2025 深圳市汇顶科技股份有限公司

13：spim_ir3

14：spim_cs1

16：pwm0_a

17：pwm0_b

18：pwm0_c

19：pwm1_a

20：pwm1_b

21：pwm1_c

24：spim_csn2

25：spim_csn3

Bit 4:0 rg_trxa_podl_func_sel

trxa_podl 引脚复用功能选择

0：trxa_podl（默认）

1：aon_gpio[0]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.2 ALON_PAD_FUNC_SEL_CTL1

Absolute Address: 0x1e0102

Base Offset: 0x2

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_gpio_alon1_func_sel		rg_gpio_alon0_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_gpio_alon1_func_sel

aon_gpio1 引脚功能选择

0：aon_gpio[3]（默认）

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_gpio_alon0_func_sel

aon_gpio0 引脚功能选择

0：aon_gpio[2]（默认）

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.3 ALON_PAD_FUNC_SEL_CTL2

Absolute Address: 0x1e0104

Base Offset: 0x4

Size: 0x2

Reset value: 0x0000

版权 © 2025 深圳市汇顶科技股份有限公司

15:13	12:8	7:5	4:0
			rg_can_rx_func_sel
			rw

Bit 15:5 reserved

Bit 7:5 reserved

Bit 4:0 rg_can_rx_func_sel

can_rx 引脚功能选择

0：can_rx（默认）

1：aon_gpio[4]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.4 SHUT_PAD_FUNC_SEL_CTL0

Absolute Address: 0x1e0106

Base Offset: 0x6

Size: 0x2

Reset value: 0x0040

15:7	6	5	4	3	2	1	0
	rg_pdm_clk1_en		rg_i2c_master_en	rg_dmic1_en	rg_dmic0_en	rg_spi_master_en	
	rw		rw	rw	rw	rw	

Bit 15:7 reserved

Bit 5 reserved

Bit 0 reserved

Bit 6 rg_pdm_clk1_en

pdm_clk1 输出复用引脚配置

在开启两路 pdm 的情况下，若需要单独输出 pdm_clk1，则可以将 CAN_TX 复用为 pdm_clk1 输出

0：不开启 pdm_clk1（默认）

1：开启 pdm_clk1

Bit 4 rg_i2c_master_en

将 I2C_SCL 和 I2C_SDA 切换为 I2C Master 引脚复用

0：不开启（默认为 I2C Slave）

1：开启

Bit 3 rg_dmic1_en

TXD_RX1 引脚切换为 pdm_data1 引脚配置

0：不切换

1：切换

Bit 2 rg_dmic0_en

TDM_RX0 引脚切换为 pdm_data0 引脚配置，TDM_BCLK 引脚切换为 pdm_clk0 引脚配置

0：不切换

1：切换

版权 © 2025 深圳市汇顶科技股份有限公司

Bit 1 rg_spi_master_en

将 SPI_CLK、SPI_CSN、SPI_MOSI、SPI_MISO 切换为 SPI Master 引脚复用

0：不开启（默认为 SPI Slave）

1：开启

5.6.3.1.5 SHUT_PAD_FUNC_SEL_CTL1

Absolute Address: 0x1e0108

Base Offset: 0x8

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_spi_ir1_func_sel		rg_spi_ir0_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_spi_ir1_func_sel

SPI_IR1 引脚功能选择

0：SPI_IR1（默认）

1：gpio[1]

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_spi_ir0_func_sel

SPI_IR0 引脚功能选择

0：SPI_IR0（默认）

1：gpio[0]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.6 SHUT_PAD_FUNC_SEL_CTL2

Absolute Address: 0x1e010a

Base Offset: 0xa

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_clk_out_func_sel		rg_irq_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_clk_out_func_sel

CLK_OUT 引脚功能选择

0：CLK_OUT（默认）

2~25：和 rg_trxb_podl_func_sel 一致

版权 © 2025 深圳市汇顶科技股份有限公司

Bit 4:0 rg_irq_func_sel

IRQ 引脚功能选择

0：IRQ（默认）

1：1PPS

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.7 SHUT_PAD_FUNC_SEL_CTL3

Absolute Address: 0x1e010c

Base Offset: 0xc

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_tdm_sync_func_sel		rg_tdm_bclk_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_tdm_sync_func_sel

TDM_SYNC 引脚功能选择

0：TDM_SYNC（默认）

1：gpio[4]

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_tdm_bclk_func_sel

TDM_BCLK 引脚功能选择

0：TDM_BCLK（默认）

1：gpio[3]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.8 SHUT_PAD_FUNC_SEL_CTL4

Absolute Address: 0x1e010e

Base Offset: 0xe

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_tdm_rx1_func_sel		rg_tdm_rx0_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_tdm_rx1_func_sel

TDM_RX1 引脚功能选择

0：TDM_RX1（默认）

版权 © 2025 深圳市汇顶科技股份有限公司

1：gpio[6]

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_tdm_rx0_func_sel

TDM_RX0 引脚功能选择

0：TDM_RX0（默认）

1：gpio[5]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.9 SHUT_PAD_FUNC_SEL_CTL5

Absolute Address: 0x1e0110

Base Offset: 0x10

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_tdm_tx1_func_sel		rg_tdm_tx0_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_tdm_tx1_func_sel

TDM_TX1 引脚功能选择

0：TDM_TX1（默认）

1：gpio[8]

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_tdm_tx0_func_sel

TDM_TX0 引脚功能选择

0：TDM_TX0（默认）

1：gpio[7]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.10 SHUT_PAD_FUNC_SEL_CTL6

Absolute Address: 0x1e0112

Base Offset: 0x12

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_i2c_ir1_func_sel		rg_i2c_ir0_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_i2c_i21_func_sel

 版权 © 2025 深圳市汇顶科技股份有限公司

I2C_IR1 引脚功能选择

0：I2C_IR1（默认）

1：gpio[10]

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_i2c_i20_func_sel

I2C_IR0 引脚功能选择

0：I2C_IR0（默认）

1：gpio[9]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.11 SHUT_PAD_FUNC_SEL_CTL7

Absolute Address: 0x1e0114

Base Offset: 0x14

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_i2c_adr1_func_sel		rg_i2c_adr0_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_i2c_adr1_func_sel

I2C_ADR1 引脚功能选择

0：I2C_ADR1（默认）

1：gpio[12]

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_i2c_i20_func_sel

I2C_ADR0 引脚功能选择

0：I2C_ADR0（默认）

1：gpio[11]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.12 SHUT_PAD_FUNC_SEL_CTL8

Absolute Address: 0x1e0116

Base Offset: 0x16

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_lin_rx_func_sel		rg_lin_tx_func_sel
	rw		rw

Bit 15:13 reserved

版权 © 2025 深圳市汇顶科技股份有限公司

Bit 7:5 reserved

Bit 12:8 rg_lin_rx_func_sel

LIN_RX 引脚功能选择

0：LIN_RX（默认）

1：gpio[14]

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_lin_tx_func_sel

LIN_TX 引脚功能选择

0：LIN_TX（默认）

1：gpio[13]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.13 SHUT_PAD_FUNC_SEL_CTL9

Absolute Address: 0x1e0118

Base Offset: 0x18

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_gpio_func_sel		rg_can_tx_func_sel
	rw		rw

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_gpio_func_sel

GPIO 引脚功能选择

0：gpio[18]（默认）

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_can_tx_func_sel

CAN_TX 引脚功能选择

0：CAN_TX（默认）

1：gpio[15]

2~25：和 rg_trxb_podl_func_sel 一致

5.6.3.1.14 SHUT_PAD_FUNC_SEL_CTL10

Absolute Address: 0x1e011a

Base Offset: 0x1a

Size: 0x2

Reset value: 0x0000

15:13	12:8	7:5	4:0
	rg_uart_tx_sel		rg_uart_rx_func_sel
	rw		rw

版权 © 2025 深圳市汇顶科技股份有限公司

Bit 15:13 reserved

Bit 7:5 reserved

Bit 12:8 rg_uart_tx_func_sel

UART_TX 引脚功能选择

0：UART_TX（默认）

1：gpio[17]

2~25：和 rg_trxb_podl_func_sel 一致

Bit 4:0 rg_uart_rx_func_sel

UART_RX 引脚功能选择

0：UART_RX（默认）

1：gpio[16]

2~25：和 rg_trxb_podl_func_sel 一致

5.7 SPIM

GE1101 支持 SPI Master 模式对外通讯，最高频率 12.5M，SPIM 的配置和使用有独立的子块协议，Header 中的数据类型需要设置为 SPIM，本章详细介绍 SPIM 的子块协议、使用方法及寄存器。

5.7.1 写寄存器

支持在一个子块中写单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x0 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，写寄存器时固定为 0x0。

reg_addr_0：访问的寄存器绝对地址。

reg_data_0：写入的寄存器值。

5.7.2 读寄存器命令

支持在一个子块中读单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x1 4bit	reg_addr_0 32bit	reg_addr_1 32bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，读寄存器时固定为 0x1。

reg_addr_0：访问的寄存器绝对地址。

5.7.3 读寄存器值

在主机发送读寄存器命令后，GE1101 会返回相应的寄存器值，支持一个或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x2 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

版权 © 2025 深圳市汇顶科技股份有限公司

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回寄存器值时固定为 0x2。

reg_addr_0：读出的寄存器绝对地址。

reg_data_0：读出的寄存器值。

5.7.4 SPIM 发送数据

支持在一次子块通讯中，包含一次或多次 SPI 总线数据发送，每次总线数据发送间隔会拉高 CSN 片选脚。

sequence 4bit	data_type=0x3 4bit	SER_DATA 8bit	trans_tot_num 8bit	vld_wr_num0 8bit	wr_data_0 8bit	wr_data_1 8bit	...	wr_data_n 8bit
				vld_wr_num1 8bit	wr_data_0 8bit	wr_data_1 8bit	...	wr_data_n 8bit
				vld_wr_numn 8bit	wr_data_0 8bit	wr_data_1 8bit	...	wr_data_n 8bit

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，发送数据时固定为 0x3。

SER_DATA：本次发送数据时选择的 SPI 片选脚

0：CSN0

1：CSN1

2：CSN2

3：CSN3

trans_tot_num：本次发送数据有多少轮 SPI 总线操作

vld_wr_numx：第 x 轮 SPI 总线包含多少字节有效数据

wr_datax：第 x 个字节数据

举例：需要 SPI 总线上发出数据 0x01 0x02 0x03，则 trans_tot_num=1，vld_wr_num0=3，wr_data0 = 0x1，wr_data1 = 0x2，wr_data2 = 0x3 即可

5.7.5 SPIM 收发数据

支持在一次子块通讯中，包含一次或多次 SPI 总线数据全双工收发，每次总线数据发送间隔会拉高 CSN 片选脚。

sequence 4bit	data_type=0x4 4bit	SER_DATA 8bit	packet id 8bit	cs_high_cycles (16bit)	trans_tot_num 8bit	vld_wr_num0 8bit	dmy_wr_num0 8bit	wr_data_0 8bit	wr_data_1 8bit	...	wr_data_n 8bit
						vld_wr_num1 8bit	dmy_wr_num1 8bit	wr_data_0 8bit	wr_data_1 8bit	...	wr_data_n 8bit
						vld_wr_num2 8bit	dmy_wr_num2 8bit	wr_data_0 8bit	wr_data_1 8bit	...	wr_data_n 8bit
						vld_wr_numn 8bit	dmy_wr_numn 8bit	wr_data_0 8bit	wr_data_1 8bit	...	wr_data_n 8bit

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，收发数据时固定为 0x4。

SER_DATA：本次发送数据时选择的 SPI 片选脚

0：CSN0

1：CSN1

2：CSN2

3：CSN3

packet_id：包序列号，用于识别返回的数据包序列

cs_high_cycles：暂未使用，0x0

版权 © 2025 深圳市汇顶科技股份有限公司

trans_tot_num：本次发送数据有多少轮 SPI 总线操作

vld_wr_numx：第 x 轮 SPI 总线需要发送的字节数

wr_datax：第 x 个发送的字节数据

dummy_wr_numx：第 x 轮 SPI 总线需要接收的字节数

举例：需要 SPI 总线上第一个 8bit 时钟发出数据 0x01，紧接着两个 8bit 时钟接收数据 2 个字节，则 trans_tot_num=1，vld_wr_num0=1，wr_data0 = 0x1，dummy_wr_num0 = 2 即可。

5.7.6 SPIM 读取返回的数据

在主机发送收发数据子块后，GE1101 会返回 SPI 总线上读取的数据子块内容如下。

sequence 4bit	data_type=0x5 4bit	packet id 8bit	rd_data_0 8bit	...	rd_data_n 8bit

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回总线数据时固定为 0x5。

packet_id：包序列号，用于标志返回的数据包对应哪次的收发数据子块序列

rd_datax：读取到的第 x 个字节数据

5.7.7 SPIM 智能中断

GE1101 支持智能中断，智能中断使用 SPI_IR 引脚，当 SPI_IR 引脚检测到中断电平之后，根据寄存器配置自动发出总线读写逻辑，并向读回的总线数据通过主机接口上报。

返回的子块格式如下。

sequence 4bit	data_type=0x6 4bit	sensor_id 4bit	rd_irq_data_len 4bit	irq_data_0 8bit	...	irq_data_n 8bit

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回中断数据时固定为 0x6。

sensor_id：sensor 索引号，用于标志返回的中断数据对应哪一组 sensor

0：csn0 对应的 sensor

1：csn1 对应的 sensor

2：csn2 对应的 sensor

3：csn3 对应的 sensor

rd_irq_data_len：中断数据长度

irq_datax：读取到的第 x 个字节数据

5.7.8 常用流程

初始化：

1） 修改寄存器 SHUT_PAD_FUNC_SEL_CTL0，将 SPI 从 Slave 模式切换到 Master 模式

2） 修改 SPI 寄存器 grp01_scph_scpol 设置 spi0 的极性相位

3） 修改 SPI 寄存器 grp01_baud 设置 spi0 的波特率

4） 修改 SPI 寄存器 SSIENR 使能 SPI

 版权 © 2025 深圳市汇顶科技股份有限公司

5） 修改 SPI 寄存器 SER 使能 spi0

6） 修改 SPI 寄存器 RXFTLR 为 0x7F，设置 RX_FIFO 大小

7） 修改 SPI 寄存器 TXFTLR 为 0x4，设置 TX_FIFO 大小

8） 修改 SPI 寄存器 CTRLR0_L 为 0x7，设置为 8bit 数据，Motorolla 格式

总线写数据：

1） 填充子块中的 SER_DATA，选择具体哪一路 SPI CSN

2） 根据传输次数，填充子块中的 trans_tot_num

3） 根据数据，填充子块中的 vld_wr_num 和 wr_data

4） 通过主机接口发送给芯片

总线读写数据：

1） 填充子块中的 SER_DATA，选择具体哪一路 SPI CSN

2） 根据传输次数，填充子块中的 trans_tot_num

3） 根据要发送数据，填充子块中的 vld_wr_num 和 wr_data

4） 根据要接收的数据长度，填充子块中的 dmy_wr_num

5） 通过主机接口发送给芯片

6） 芯片返回数据包到主机接口

7） 解析 Header 及子块，取出 packet_id 和 data

智能中断：

1） 修改 SPI 寄存器 grp0_ctrl，打开及选择中断后的读写逻辑模式

2） 修改 SPI 寄存器 sensor_int_ctrl，设置中断的电平极性和类型

3） 修改 SPI 寄存器 grp0_vld_wr_num，设置写逻辑中的数据长度

4） 修改 SPI 寄存器 grp0_dmy_wr_num，设置读逻辑中的数据长度

5） 修改 SPI 寄存器 grp0_wr_data，设置写逻辑中的数据内容

6） 触发中断

7） 芯片返回读出的数据到主机接口

8） 解析 Header 和子块，取出 sensor_id，rd_irq_data_len 和 data

5.7.9 参考实现

参考 SDK 中如下接口

uint8_t ge_api_spi_config(uint8_t node_index, uint8_t spi_index, uint32_t speed, uint8_t cpha, uint8_t cpol, uint16_t time_out_ms)

uint8_t ge_api_spi_sensor_irq_config(uint8_t node_index, ge_spi_irq_config_t spi_irq_config, uint16_t time_out_ms)

uint8_t ge_api_spi_tx(uint8_t node_index, uint8_t spi_index, uint8_t *tx_data, uint8_t data_len, uint16_t time_out_ms)

uint8_t ge_api_spi_txrx(uint8_t node_index, uint8_t spi_index, uint8_t *tx_data,uint8_t cs_high_cycle, uint8_t *rx_data, uint8_t tx_len, uint8_t rx_len, uint16_t time_out_ms)

uint8_t ge_api_spi_sensor_get_irq_data(uint8_t *sensor_index, uint8_t *data, uint8_t *len)

5.7.10 寄存器

5.7.10.1.1 grp01_scph_scpol

Absolute Address: 0x1f290a

Base Offset: 0xa

Size: 0x2

Reset value: 0x0000

15:2	1	0
	rg_grp01_scpol	rg_grp01_scph
	rw	rw

Bit 15:2 reserved

Bit 1 rg_grp01_scpol

SPI0 的 CPOL 极性

0：空闲时时钟处于低电平

1：空闲时时钟处于高电平

Bit 0 rg_grp01_scph

SPI0 的 CPHA 相位

0：上升沿采样数据

1：下降沿采样数据

5.7.10.1.2 grp01_baud

Absolute Address: 0x1f290c

Base Offset: 0xc

Size: 0x2

Reset value: 0x0000

15:0
rg_grp01_baud
rw

Bit 15:0 rg_grp01_baud

SPI0 的波特率

波特率计算公式：fspi = fref / baud

5.7.10.1.3 SSIENR

Absolute Address: 0x1f2808

Base Offset: 0x8

Size: 0x2

Reset value: 0x0000

15:1	0
	SSI_EN
	Rw

Bit 15:1 reserved

Bit 0 SSI_EN

SPI 使能开关

0：关闭 SPI

1：开启 SPI

5.7.10.1.4 SER

Absolute Address: 0x1f2810

Base Offset: 0x10

Size: 0x2

Reset value: 0x0000

15:4	3	2	1	0
	SER3	SER2	SER1	SER0
	rw	rw	rw	Rw

Bit 15:4 reserved

Bit 3 SER3

SPI 通道 3 使能开关

0：关闭

1：开启

SER2~SER0 同上

5.7.10.1.5 TXFTLR

Absolute Address: 0x1f2818

Base Offset: 0x18

Size: 0x2

Reset value: 0x0000

15:7	6:0
	TFT
	rw

Bit 15:7 reserved

Bit 6:0 TFT

发送 FIFO 等级

5.7.10.1.6 RXFTLR

Absolute Address: 0x1f281c

Base Offset: 0x1c

Size: 0x2

Reset value: 0x0000

15:7	6:0
	RFT
	rw

Bit 15:7 reserved

Bit 6:0 RFT

接收 FIFO 等级

5.7.10.1.7 CTRLR0_L

Absolute Address: 0x1f2800

Base Offset: 0x0

Size: 0x2

Reset value: 0x0000

15:6	5:4	3:0
	FRF	DFS
	rw	rw

Bit 15:6 reserved

Bit 5:4 FRF

帧格式

0： motorolla SPI

1： TI SSP

2： Nationnal Microwire

Bit 3:0

数据长度

3： 4bit

4： 5bit

5： 6bit

6： 7bit

7： 8bit

8： 9bit

9： 10bit

10： 11bit

11： 12bit

12： 13bit

13： 14bit

14： 15bit

15： 16bit

版权 © 2025 深圳市汇顶科技股份有限公司

5.7.10.1.8 grp0_ctrl

Absolute Address: 0x1f2908

Base Offset: 0x8

Size: 0x2

Reset value: 0x0000

15:2	1	0
	rg_grp0_sel_wr_rd_sensor	rg_grp0_en
	rw	rw

Bit 15:2 reserved

Bit 1 rg_grp0_sel_wr_rd_sensor

SPI 通道 0 的智能中断读写逻辑

0：读

1：写

Bit 0 rg_grp0_en

SPI 通道 0 的智能中断开关

0：关闭

1：开启

5.7.10.1.9 grp0_vld_wr_num

Absolute Address: 0x1f2910

Base Offset: 0x10

Size: 0x2

Reset value: 0x0000

15:8	7:0
	rg_grp0_vld_wr_num0
	rw

Bit 15:8 reserved

Bit 7:0 rg_grp0_vld_wr_num0

若智能中断的总线逻辑是写操作，此处设置为总线上要写的数据长度

若智能中断的总线逻辑是读操作，此处设置为读数据之前总线上写的数据长度

5.7.10.1.10 grp0_dmy_wr_num

Absolute Address: 0x1f2912

Base Offset: 0x12

Size: 0x2

Reset value: 0x0000

15:8	7:0
	rg_grp0_dmy_wr_num0
	rw

Bit 15:8 reserved

版权 © 2025 深圳市汇顶科技股份有限公司

Bit 7:0 rg_grp0_dmy_wr_num0

若智能中断的总线逻辑是写操作，设置为 0

若智能中断的总线逻辑是读操作，此处设置为读数据的长度

5.7.10.1.11 grp0_wr_data_0

Absolute Address: 0x1f2914

Base Offset: 0x14

Size: 0x2

Reset value: 0x0000

15:8	7:0
rg_grp0_wr_data_0	rg_grp0_wr_data_1
rw	rw

Bit 15:8 rg_grp0_wr_data_0

智能中断读写逻辑中输出的数据 0

Bit 7:0 rg_grp0_wr_data_1

智能中断读写逻辑中输出的数据 1

grp0_wr_data1~grp0_wr_data13 类同此寄存器

grp1~grp7 类同上述智能中断寄存器

5.7.10.1.12 sensor_int_ctrl

Absolute Address: 0x1f2a30

Base Offset: 0x130

Size: 0x2

Reset value: 0x0000

15:4	3	2	1	0
	rg_int3_edge_select	rg_int2_edge_select	rg_int1_edge_select	rg_int0_edge_select
	rw	rw	rw	rw

Bit 15:4 reserved

Bit 3 rg_int3_edge_select

SPI3 的智能中断极性

0：高电平触发

1：低电平触发

rg_int2_edge_select~ rg_int0_edge_select 同上

SPI0 的 CPHA 相位

0：上升沿采样数据

1：下降沿采样数据

5.8 I2CM

GE1101 支持 I2C Master 模式对外通讯，最高频率 1M，I2CM 的配置和使用有独立的子块协议，Header 中的数据类型需要设置为 I2CM，本章详细介绍 I2CM 的子块协议、使用方法及寄存器。

5.8.1 写寄存器

支持在一个子块中写单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x0 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，写寄存器时固定为 0x0。

reg_addr_0：访问的寄存器绝对地址。

reg_data_0：写入的寄存器值。

5.8.2 读寄存器命令

支持在一个子块中读单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x1 4bit	reg_addr_0 32bit	reg_addr_1 32bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，读寄存器时固定为 0x1。

reg_addr_0：访问的寄存器绝对地址。

5.8.3 读寄存器值

在主机发送读寄存器命令后，GE1101 会返回相应的寄存器值，支持一个或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x2 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回寄存器值时固定为 0x2。

reg_addr_0：读出的寄存器绝对地址。

reg_data_0：读出的寄存器值。

5.8.4 I2CM 发送数据

支持在一次子块通讯中，对一个 I2C 外设进行一次或多次 I2C 总线数据发送。

sequence 4bit	idata_type=0x3 4bit	i2c_base_addr 8bit	packet_tot_num 8bit	wr_num0_num 8bit	wr_num0_data_0 8bit	.....	wr_num0_data_n 8bit

wr_num1_num 8bit	wr_num1_data_0 8bit	.....	wr_num1_data_n 8bit	.....	wr_numN_num 8bit	wr_numN_data_0 8bit	.....	wr_numN_data_n 8bit

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，发送数据时固定为 0x3。

I2c_base_addr：i2c 外设的 7bit 地址

Packet_total_num：本次发送数据有多少轮 I2C 总线操作

Wr_numx_num：第 x 轮 I2C 总线包含多少字节有效数据

版权 © 2025 深圳市汇顶科技股份有限公司

wr_numx_data_x：第 x 个字节数据

5.8.5 读取 I2CM 数据命令

支持在一次子块通讯中，包含一次或多次 I2C 总线数据读操作。

sequence 4bit	data_type=0x4 4bit	packet_id 8bit	i2c_base_addr 8bit	rd_packet_tot_num 8bit	rd_data_num0 8bit	wr_data_len0 8bit	wr_data0_0 8bit	.....

wr_data0_n 8bit	rd_data_num1 8bit	wr_data_len1 8bit	wr_data1_0 8bit	.....	wr_data1_n 8bit	.....

rd_data_numN 8bit	wr_data_lenN 8bit	wr_dataN_0 8bit	.....	wr_dataN_n 8bit

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，读取数据命令时固定为 0x4。

packet_id：包序列号，用于识别返回的数据包序列

i2c_base_addr：i2c 设备的 7bit 地址

rd_packet_tot_num：本次发送数据有多少轮 I2C 总线操作

rd_data_numx：第 x 轮 I2C 总线需要读取的字节数

wr_data_lenx：第 x 轮 I2C 总线读取前需要发送的数据字节数

wr_data_x_x：第 x 轮 I2C 总线读取前需要发送的第 x 个数据

5.8.6 I2CM 读取返回的数据

在主机发送读取数据子块后，GE1101 会返回 I2C 总线上读取的数据子块内容如下。

sequence 4bit	data_type=0x5 4bit	packet_id(same) 8bit	i2c_base_addr 8bit	rd_data_0 8bit	rd_data_1 8bit	...	rd_data_n 8bit

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回总线数据时固定为 0x5。

packet_id：包序列号，用于标志返回的数据包对应哪次的收发数据子块序列

i2c_base_addr：读取的 i2c 设备 7bit 地址

rd_datax：读取到的第 x 个字节数据

5.8.7 I2CM 智能中断

GE1101 支持智能中断，智能中断使用 I2C_IR 引脚，当 I2C_IR 引脚检测到中断电平之后，根据寄存器配置自动发出总线读写逻辑，并向读回的总线数据通过主机接口上报。

返回的子块格式如下。

sequence 4bit	data_type=0x6 4bit	i2c_base_addr 8bit	rd_irq_data_len 8bit	irq_data_0 8bit	irq_data_1 8bit	...	irq_data_n 8bit

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回中断数据时固定为 0x6。

I2c_base_addr：读取的数据的 I2C 外设 7bit 地址

rd_irq_data_len：中断数据长度

irq_datax：读取到的第 x 个字节数据

版权 © 2025 深圳市汇顶科技股份有限公司

5.8.8 常用流程

初始化：

1） 修改寄存器 SHUT_PAD_FUNC_SEL_CTL0，将 I2C 从 Slave 模式切换到 Master 模式

2） 修改 I2C 寄存器 IC_CON_L 设置 i2c 的速率

总线写数据：

1） 填充子块中的 i2c_base_addr，确认访问的 i2c 设备 7bit 地址

2） 根据传输次数，填充子块中的 packet_tot_num

3） 根据数据，填充子块中的 wr_num_num 和 wr_num_data

4） 通过主机接口发送给芯片

总线读数据：

1） 填充子块中的 i2c_base_addr，确认访问的 i2c 设备 7bit 地址

2） 根据传输次数，填充子块中的 rd_packet_tot_num

3） 根据要发送数据，填充子块中的 wr_data_len 和 wr_data

4） 根据要接收的数据长度，填充子块中的 rd_data_num

5） 通过主机接口发送给芯片

6） 芯片返回数据包到主机接口

7） 解析 Header 及子块，取出 packet_id、i2c_base_addr 和 data

智能中断：

1） 修改 I2C 寄存器 sensor_irq_mask，打开需要使能的那一路智能中断

2） 修改 I2C 寄存器 sensor_irq_trig_mode，设置使用的那一路中断的电平极性和类型

3） 修改 I2C 寄存器 grp0_en，使能智能中断

4） 修改 I2C 寄存器 grp0_irq_index，设置该组智能中断对应哪一路中断引脚

5） 修改 I2C 寄存器 grp0_irq_con，设置触发中断后的读写逻辑

6） 修改 I2C 寄存器 grp0_wr_data，设置触发中断后的写数据内容

7） 触发中断

8） 芯片返回读出的数据到主机接口

9） 解析 Header 和子块，取出 i2c_base_addr，rd_irq_data_len 和 data

5.8.9 参考实现

参考 SDK 中如下接口

uint8_t ge_api_i2c_set_config(uint8_t node_index, uint8_t enable, uint8_t speed)

 版权 © 2025 深圳市汇顶科技股份有限公司

uint8_t ge_api_i2c_write_sensor_data(uint8_t node_index, uint8_t sensor_addr, uint8_t reg_addr, uint8_t reg_data, uint16_t time_out_ms)

uint8_t ge_api_i2c_read_sensor_data(uint8_t node_index, uint8_t sensor_addr, uint8_t reg_addr, uint8_t *reg_data, uint16_t time_out_ms)

uint8_t ge_api_i2c_sensor_irq_config(uint8_t node_index, ge_i2c_irq_config_t i2c_irq_config, uint16_t time_out_ms)

uint8_t ge_api_i2c_sensor_get_irq_data(uint8_t *sensor_addr, uint8_t *data, uint8_t *len)

5.8.10 寄存器

5.8.10.1.1 IC_CON_L

Absolute Address: 0x1f2200

Base Offset: 0x0

Size: 0x2

Reset value: 0x0025

15:6	5	4:3	2:1	0
	IC_RESTART_EN		SPEED	MASTER_MODE
	rw		rw	rw

Bit 15:6 reserved

Bit 4:3 reserved

Bit 5 IC_RESTART_EN

I2C 时序中是否支持 RESTART

0：不支持（使用 STOP+START）

1：支持（使用 RESTART，默认）

Bit 2:1 SPEED

1：标准（100k）

2：快速（默认 400k~1000k）

3：高速（3.4M）

Bit 0 MASTER_MODE

0：关闭 I2C master 模式

1：开启 I2C master 模式（默认）

5.8.10.1.2 sensor_irq_mask

Absolute Address: 0x1f230e

Base Offset: 0xe

Size: 0x2

Reset value: 0x0000

15:8	7	6	5	4	3	2	1	0
	rg_sensor_irq_mask[7:0]							
	rw							

Bit 15:8 reserved

Bit 7 rg_sensor_irq_mask[7]

0：第 7 路 I2CM 智能中断掩码关

1：第 7 路 I2CM 智能中断掩码开

Bit 6:0 类同 Bit 7，控制每一个的智能中断掩码

5.8.10.1.3 sensor_irq_trig_mode

Absolute Address: 0x1f2310

Base Offset: 0x10

Size: 0x2

Reset value: 0x0000

15:14	13:12	11:10	9:8	7:6	5:4	3:2	1:0
rg_irq_trig_mode[7:0]							
rw	rw	rw	rw	Rw	rw	rw	rw

Bit 15:14 rg_irq_trig_mode[7]

第 7 路 I2CM 智能中断触发极性和模式

0：上升沿触发

1：下降沿触发

2：高电平触发

3：低电平触发

Bit 13:0 类同 Bit 15:14，控制每一个的智能中断触发的极性和模式

5.8.10.1.4 grp0_en

Absolute Address: 0x1f2312

Base Offset: 0x12

Size: 0x2

Reset value: 0x0000

15:1	0
	rg_grp0_auto_irq_en
	rw

Bit 15:1 reserved

Bit 0 rg_grp0_auto_irq_en

第 0 组智能中断使能开关

0：关闭

1：打开

5.8.10.1.5 grp0_irq_index

Absolute Address: 0x1f2314

Base Offset: 0x14

Size: 0x2

Reset value: 0x0000

版权 © 2025 深圳市汇顶科技股份有限公司

15:4	3:0
	rg_grp0_auto_irq_index
	rw

Bit 15:4 reserved

Bit 3:0 rg_grp0_auto_irq_index

第 0 组智能中断配置适用于那个智能中断引脚

0：I2C_IR0

1：I2C_IR1

……

7：I2C_IR7

5.8.10.1.6 grp0_irq_con

Absolute Address: 0x1f2316

Base Offset: 0x16

Size: 0x2

Reset value: 0x0000

15:12	11:8	7:0
rg_grp0_auto_irq_rd_num	rg_grp0_auto_irq_wr_num	rg_grp0_auto_irq_i2c_base_addr
rw	rw	rw

Bit 15:12 rg_grp0_auto_irq_rd_num

第 0 组智能中断配置读写逻辑中需要读的数据长度

Bit 11:8 rg_grp0_auto_irq_wr_num

第 0 组智能中断配置读写逻辑中读之前需要写的数据长度

Bit 7:0 rg_grp0_auto_irq_i2c_base_addr

第 0 组智能中断配置读写逻辑中的外设 7bit 地址

5.8.10.1.7 grp0_wr_data_0

Absolute Address: 0x1f2318

Base Offset: 0x18

Size: 0x2

Reset value: 0x0000

15:8	7:0
rg_grp0_auto_irq_wr_data1	rg_grp0_auto_irq_wr_data0
rw	rw

Bit 15:8 rg_grp0_auto_irq_wr_data1

第 0 组智能中断配置读写逻辑中写的数据 1

Bit 7:0 rg_grp0_auto_irq_wr_data0

第 0 组智能中断配置读写逻辑中写的数据 0

grp0_wr_data1~3 类同 grp0_wr_data_0

grp1~7 的配置类同 grp0

版权 © 2025 深圳市汇顶科技股份有限公司

5.9 CAN

GE1101 支持 CAN（FD）模式对外通讯，支持 Bosch 和 ISO 标准，CAN 的配置和使用有独立的子块协议，Header 中的数据类型需要设置为 CAN，本章详细介绍 CAN 的子块协议、使用方法及寄存器。

5.9.1 写寄存器

支持在一个子块中写单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x0 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，写寄存器时固定为 0x0。

reg_addr_0：访问的寄存器绝对地址。

reg_data_0：写入的寄存器值。

5.9.2 读寄存器命令

支持在一个子块中读单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x1 4bit	reg_addr_0 32bit	reg_addr_1 32bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，读寄存器时固定为 0x1。

reg_addr_0：访问的寄存器绝对地址。

5.9.3 读寄存器值

在主机发送读寄存器命令后，GE1101 会返回相应的寄存器值，支持一个或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x2 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回寄存器值时固定为 0x2。

reg_addr_0：读出的寄存器绝对地址。

reg_data_0：读出的寄存器值。

5.9.4 CAN 发送数据

支持在一次子块通讯中，对 CAN 总线发起一帧或多帧 CAN 数据传输。

sequence 4bit	data_type=0x3 4bit	packet_num 8bit	frame0_len 8bit	frame0_data0 8bit	frame0_data1 8bit	...	frame1_len 8bit	frame1_data0 8bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，发送数据时固定为 0x3。

packet_num：需要发送的数据帧数

版权 © 2025 深圳市汇顶科技股份有限公司

framex_len：第 x 帧的数据长度

framex_datax：第 x 帧第 x 个数据

offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	DATA 1 (DLC<1 then free)								ID[2:0]			RSVD	ESI	RSVD			ID[10:3]								IDE	RRS/RTR	FDF	BRS	DLC			
0x001	DATA 5 (DLC<5 then free)								DATA 4 (DLC<4 then free)								DATA 3 (DLC<3 then free)								DATA 2 (DLC<2 then free)							
0x002	DATA 9 (DLC<9 then free)								DATA 8 (DLC<8 then free)								DATA 7 (DLC<7 then free)								DATA 6 (DLC<6 then free)							
0x003	DATA 13 (DLC<10 then free)								DATA 12 (DLC<9 then free)								DATA 11 (DLC<9 then free)								DATA 10 (DLC<9 then free)							
0x004	DATA 17 (DLC<11 then free)								DATA 16 (DLC<10 then free)								DATA 15 (DLC<10 then free)								DATA 14 (DLC<10 then free)							
0x005	DATA 21 (DLC<11 then free)								DATA 20 (DLC<11 then free)								DATA 19 (DLC<11 then free)								DATA18 (DLC<11 then free)							
0x006	DATA 25 (DLC<12 then free)								DATA 24 (DLC<11 then free)								DATA 23 (DLC<11 then free)								DATA 22 (DLC<11 then free)							
...	...								...								...								...							
0x00F	DATA 61 (DLC<15 then free)								DATA 60 (DLC<15 then free)								DATA 59 (DLC<15 then free)								DATA 58 (DLC<15 then free)							
0x010	FIDX								DATA 64 (DLC<15 then free)								DATA 63 (DLC<15 then free)								DATA 62 (DLC<15 then free)							

offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	ID[12:5]								ID[20:13]								ID[28:21]								IDE	SRR/RTR	FDF	BRS	DLC			
0x001	DATA 3 (DLC<3 then free)								DATA 2 (DLC<2 then free)								DATA 1 (DLC<1 then free)								ID[4:0]					RRS/RTR	ESI	RSVD
0x002	DATA 7 (DLC<9 then free)								DATA 6 (DLC<8 then free)								DATA 5 (DLC<7 then free)								DATA 4 (DLC<6 then free)							
0x003	DATA 11 (DLC<10 then free)								DATA 10 (DLC<9 then free)								DATA 9 (DLC<9 then free)								DATA 8 (DLC<9 then free)							
0x004	DATA 15 (DLC<11 then free)								DATA 14 (DLC<10 then free)								DATA 13 (DLC<10 then free)								DATA 12 (DLC<10 then free)							
0x005	DATA 19 (DLC<11 then free)								DATA 18 (DLC<11 then free)								DATA 17 (DLC<11 then free)								DATA16 (DLC<11 then free)							
...	...								...								...								...							
0x00F	DATA 59 (DLC<15 then free)								DATA 58 (DLC<15 then free)								DATA 57 (DLC<15 then free)								DATA 56 (DLC<15 then free)							
0x010	DATA 64 (DLC<15 then free)								DATA 62 (DLC<15 then free)								DATA 61 (DLC<15 then free)								DATA 60 (DLC<15 then free)							
0x011	free								free								FIDX								DATA 64 (DLC<15 then free)							

frame data 的格式如上，标准帧和扩展帧有细微差异，其中各字段内容参考 CAN 总线协议，DLC 即数据长度的格式如下。

Frames	Number of bytes	Data Length Code			
		DLC3	DLC2	DLC1	DLC0
Classic and FD frames	0	0	0	0	0
	1	0	0	0	1
	2	0	0	1	0
	3	0	0	1	1
	4	0	1	0	0
	5	0	1	0	1
	6	0	1	1	0
	7	0	1	1	1
	8	1	0	0	0
Classic frames	8	1	0/1	0/1	0/1
FD frames	12	1	0	0	1
	16	1	0	1	0
	20	1	0	1	1
	24	1	1	0	0
	32	1	1	0	1
	48	1	1	1	0
	64	1	1	1	1

5.9.5 读取 CAN 数据

当总线上有 CAN 数据响应时，GE1101 会返回 CAN 总线上的数据子块内容如下。

sequence 4bit	data_type=0x4 4bit	packet_num 8bit	frame0_len 8bit	frame0_data0 8bit	frame0_data1 8bit	...	frame1_len 8bit	frame1_data0 8bit	...

sequence：序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回总线数据时固定为 0x4。

packet_num：返回的数据帧数

framex_len：第 x 帧的数据长度

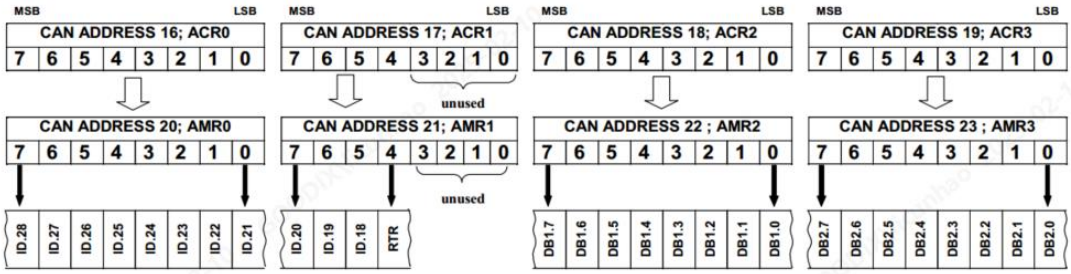
framex_datax：第 x 帧的第 x 个数据

返回数据包的 frame_data 的格式和主控写数据包的格式基本一致，但是会多一个 byte 对应 FIDX，也就是接收 CAN frame 的时候，所匹配的 Filter 的 index。目前 CAN-FD 支持 8 组不同的 Filter。CAN-FD 会返回第一个匹配上的 Filter 的 index 并写入 FIDX。因此，相同的 can frame 在返回数据包中，frame_len 会比下发的时候多 1。

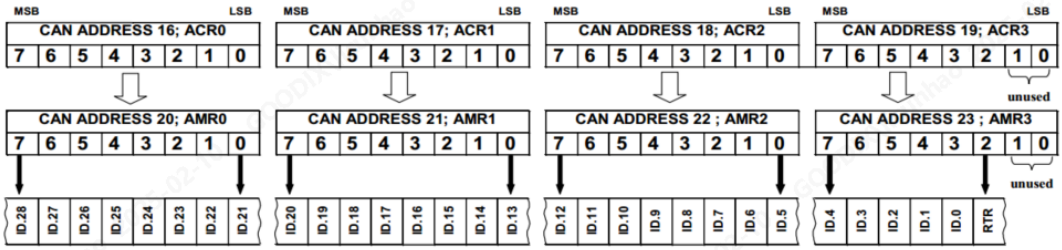
5.9.6 过滤器

GE1101 的 CAN 设备支持 8 组过滤器，对接收的信息进行过滤，过滤器有单过滤器和双过滤器两种模式，现在分别介绍。

5.9.6.1 单过滤器

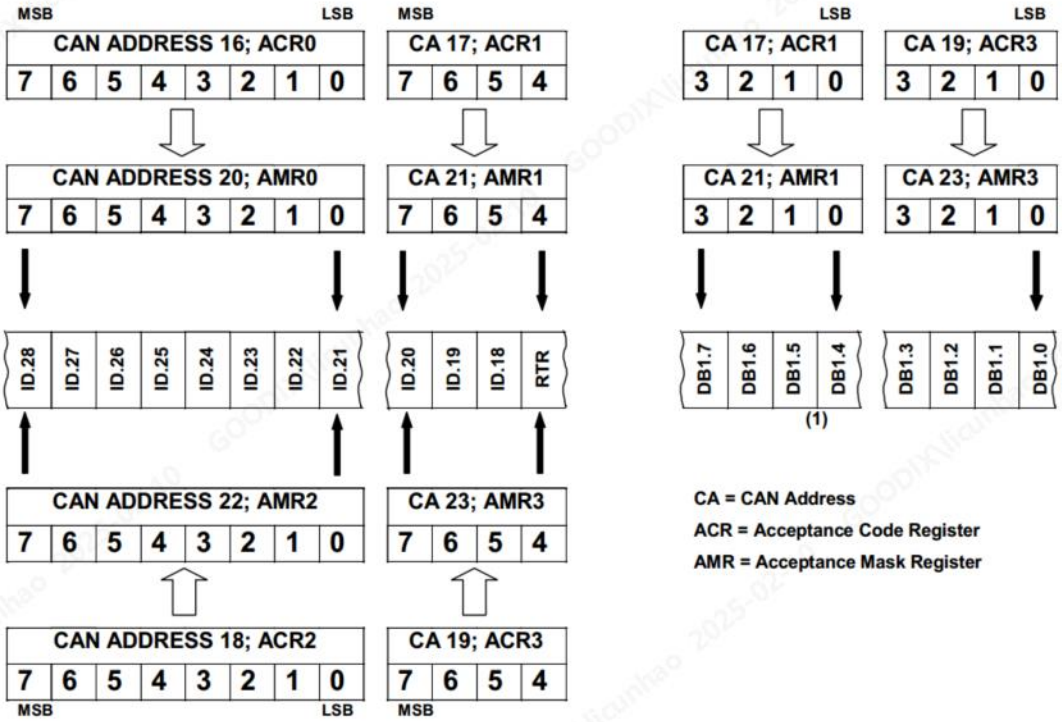


标准帧单过滤器原理如上，设置寄存器 ACR0~ACR3，AMR0~AMR3，分别过滤 PID[28:18]、RTR、DB1 和 DB2，其中 AMR 寄存器为过滤掩码，bit 为 0 的部分需要检查，bit 为 1 的部分则不关心，举例：当 AMR0 为 0xFE，则会检查 PID[21]的值是否和 ACR0[0]一致。

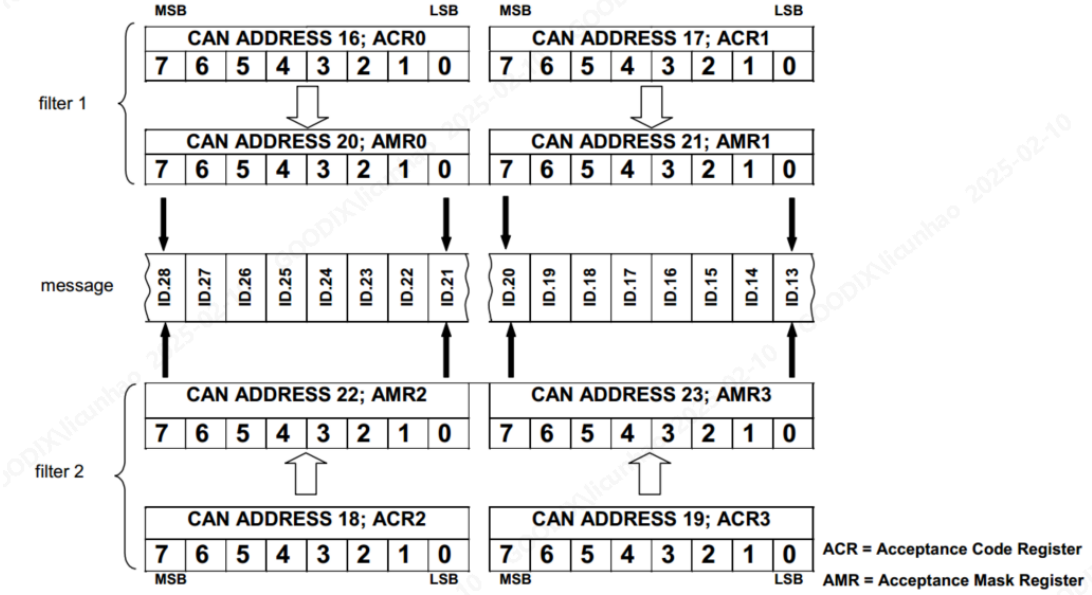


扩展帧单过滤器原理如上，其过滤原理一致。

5.9.6.2 双过滤器



标准帧双过滤器原理如上，过滤原理类似，只是减少过滤的信息类型，增加可过滤的种类，将 1 个过滤器拆分成 2 个小过滤器使用。



扩展帧双过滤器原理如上。

5.9.7 常用流程

初始化：

1. 修改寄存器 CLK_MUX，选择 CAN 模块的参考时钟
2. 修改寄存器 CLK_CAN_REG，设置 CAN 模式的参考时钟分频系数
3. 修改寄存器 rg_can_status，打开 CAN 模块
4. 修改 CAN 寄存器 rg_cmd_mr，使 CAN 模块进入 reset 模式
5. 修改 CAN 寄存器 rg_tdcr_txscnt，设置 CAN 的发送延时兼容参数
6. 修改 CAN 寄存器 can_local_reg，配置 CAN 的总线特性
7. 修改 CAN 寄存器 rg_fdsta_con，配置 CAN 的总线模式
8. 修改 CAN 寄存器 rg_nbt1、rg_nbt2、rg_dbt、rg_sspp_dbtu 和 rg_btr1_btr0 配置 CAN 设备的总线波特率
9. 修改 CAN 寄存器 rg_amr1_0、rg_amr3_2 设置滤波掩码为 0xFFFF（不过滤）
10. 修改 CAN 寄存器 rg_cmd_mr，使 CAN 模块退出 reset 模式

发送数据：

1. 根据需求，填充子块中的 packet_num、frame_len、frame_data
2. 通过主机通讯接口发送给芯片

读取数据：

1. 芯片收到总线上的 CAN 数据
2. 通过主机通讯接口发送给主机

3. 解析 Header、子块数据，获取 packet_num、frame_len、frame_data

4. 解析 frame_data 获取 CAN 数据

设置过滤器

1. 修改 CAN 寄存器 rg_cmd_mr，选择单滤波器或者双过滤器

2. 修改 CAN 寄存器 rg_txbsel_acfsel，选择当前过滤器组

3. 修改 CAN 寄存器 rg_acr、rg_amr 设置过滤条件

5.9.8 参考实现

参考 SDK 中如下接口

uint8_t ge_api_can_set_config(uint8_t node_index, ge_can_config_t *config, uint16_t time_out_ms)

uint8_t ge_api_can_write_sensor_data(uint8_t node_index, uint8_t pid_type, uint32_t pid, uint8_t can_type, uint8_t can_RTR, uint8_t brs, uint8_t *data, uint8_t len, uint16_t time_out_ms)

uint8_t ge_process_can_message(uint8_t packet_num, uint8_t *data, uint16_t data_len)

5.9.9 寄存器

5.9.9.1.1 CLK_MUX

Absolute Address: 0x1e0006

Base Offset: 0x6

Size: 0x2

Reset value: 0x0018

15:8	7:6	5	4	3	2:0
	rg_rec_clk_out_sel	rg_can_clk_sel	rg_pcs_tx_clk_sel	rg_tdm_bclk_en	
	rw	rw	rw	rw	

Bit 15:8 reserved

Bit 2:0 reserved

Bit 7:6 rg_rec_clk_out_sel

CLK_OUT 引脚输出选择

0：25M 方波输出

1：xMII 25M 方波输出

Bit 5 rg_can_clk_sel

CAN 模块主频选择

0：外部晶振频率

1：10 倍外部晶振频率

Bit 4 rg_pcs_tx_clk_sel

Pcs_tx_clk 输出相位选择

0：xMII 66M

1：xMii 66M 反相

Bit 3 rg_tdm_blck_en

Tdm_bclk 输出使能

0：关闭

1：开启

5.9.9.1.2 CLK_CAN_REG

Absolute Address: 0x1e0018

Base Offset: 0x18

Size: 0x2

Reset value: 0x001a

15:5	4	3:0
	rg_pll250m_clk_en	rg_div_can_factor
	rw	rw

Bit 15:5 reserved

Bit 4 rg_pll250m_clk_en

10 倍外部晶振频率开关

0：关闭

1：开启

Bit 3:0 rg_div_can_factor

10 倍外部晶振频率分频系数，当选择 10 倍外部晶振作为 CAN 模块主时钟源时，CAN 的主频由此分频系数决定，举例：外部晶振为 25M 时，选择 10 倍外部晶振作为 CAN 主时钟源，分频系数为 5，则 CAN 的主时钟频率为 50M。

5.9.9.1.3 rg_can_status

Absolute Address: 0x1f136e

Base Offset: 0x6e

Size: 0x2

Reset value: 0x0001

15:1	0
	rg_can_en
	rw

Bit 15:1 reserved

Bit 0 rg_can_en

CAN 模块开关。

0：关闭

1：打开

5.9.9.1.4 rg_cmd_mr

Absolute Address: 0x1f2400

Base Offset: 00

Size: 0x2

Reset value: 0x0000

 版权 © 2025 深圳市汇顶科技股份有限公司

15:6	5:3	2	1	0
	rg_receive_fifo_trigger	rg_reset_mode		rg_filter_mode
	rw	rw		rw

Bit 15:6 reserved

Bit 1 reserved

Bit 5:3 rg_receive_fifo_trigger

CAN 模块接收缓存触发等级。

0：1 word

1：4 words

2：8 words

3：16 words

4：32 words

5：64 words

Bit 2 rg_reset_mode

复位模式使能，配置 CAN 的各项参数前，需要进入复位模式，配置完成后退出复位模式方可正常使用 CAN 模块。

0：退出复位模式

1：进入复位模式

Bit 0 rg_filter_mode

CAN 模块接收过滤器配置

0：双过滤器

1：单过滤器

5.9.9.1.5 rg_tdcr_txscnt

Absolute Address: 0x1f2432

Base Offset: 0x32

Size: 0x2

Reset value: 0x0012

15	14:8	7:0
rg_tdcen	rg_tdco	rg_txscnt
rw	Rw	r

Bit 15 rg_tdcen

收发器延迟兼容功能开关，在 CAN_TX 接上收发器之后，从收发器上返回的 CAN_TX 数据的延时兼容功能，CANFD 模式下推荐打开

0：关闭

1：开启

Bit 14:8 rg_tdco

收发器延时兼容偏移值。

Bit 7:0 rg_txscnt

CAN 模块 TX 缓存空间（只读）

 版权 © 2025 深圳市汇顶科技股份有限公司

5.9.9.1.6 can_local_reg

Absolute Address: 0x1f243C

Base Offset: 0x0

Size: 0x2

Reset value: 0x0001

15:1	0
	rg_transmit_type
	rw

Bit 15:1 reserved

Bit 0 rg_tansmit_type

CAN 发送数据类型，推荐使用正常模式

0：正常模式

1：single shot 模式（不重发模式）

5.9.9.1.7 rg_fdsta_con

Absolute Address: 0x1f2424

Base Offset: 0x24

Size: 0x2

Reset value: 0x000d

15:5	4	3	2	1	0
	rg_dar	rg_iso	rg_extbt	rg_brksen	rg_fden
	rw	rw	rw	rw	Rw

Bit 15:5 reserved

Bit 4 rg_dar

自动重发功能使能

0：开启

1：关闭

Bit 3 rg_iso

CANFD 协议标准选择

0：bosch CANFD 标准

1：ISO CANFD 标准

Bit 2 rg_extbt

波特率额外寄存器配置

0：使用寄存器 rg_btr1_btr0 配置波特率

1：使用寄存器 rg_nbt、rg_dbt、rg_sspp_dbtu 配置波特率（适用于 CANFD 的仲裁域和数据域速率不同的情况）

Bit 1 rg_brksen

波特率变化支持，如果 CANFD 的仲裁域和数据域速率不同，需要打开此位

0：不支持

1：支持

Bit 0 rg_fden

CANFD 支持开关，使用 CANFD 的情况下打开此位

0：Classic CAN 格式

1：CANFD 格式

5.9.9.1.8 rg_btr1_btr0

Absolute Address: 0x1f2406

Base Offset: 0x06

Size: 0x2

Reset value: 0x0000

15	14:12	11:8	7:6	5:0
rg_sam	rg_tseg2	rg_tseg1	rg_sjw	rg_brp
rw	rw	rw	rw	rw

Bit 15 rg_sam

总线采样次数

0：1 次

1：3 次

Bit 14:12 rg_tseg2

波特率参数中的 tseg2

Bit 11:8 rg_tseg1

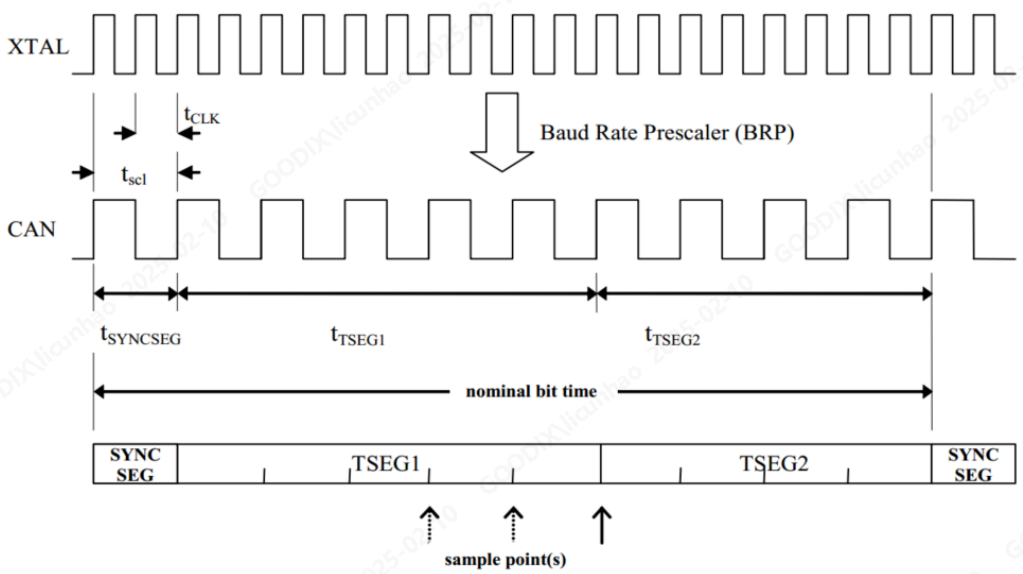
波特率参数中的 tseg1

Bit 7:6 rg_sjw

波特率参数中的 Sync Jump Width

Bit 5:0 rg_brp

波特率参数中的分频系统



Bit time 计算方式如上图所示，举例：假设 CAN 模块主时钟为 25M，要满足波特率 500K 的话，bit time

= 25000000 /500000/ (brp + 1) = (sjw + 1) + (tseg1 + 1) +(tseg2 + 1)

brp 选择 4 的情况下，bit time = 10，则 sjw + tseg1 + tseg2 = 7，可以选择 sjw = 0，tseg1 = 5，tseg2 = 2，得出 brp = 4，sjw = 0，tseg1 = 5，tseg2 = 2 的配置参数，当然，实际参数的选择和波特率以及总线情况等相关，需要具体情况具体分析。

5.9.9.1.9 rg_nbt1

Absolute Address: 0x1f241c

Base Offset: 0x1c

Size: 0x2

Reset value: 0x806

15:10	9:0
rg_nseg1[5:0]	rg_nbrp
rw	rw

Bit 15:10 rg_nseg1[5:0]

CANFD 仲裁域波特率参数 tseg1 低 6 位

Bit 9:0 rg_nbrp

CANFD 仲裁域波特率参数 brp

5.9.9.1.10 rg_nbt2

Absolute Address: 0x1f241e

Base Offset: 0x1e

Size: 0x2

Reset value: 0x208

15:9	8:2	1:0
rg_nsjw[7:1]	rg_nseg2	rg_nseg1[7:6]
rw	rw	rw

Bit 15:9 rg_nsjw

CANFD 仲裁域波特率参数 sjw

Bit 8:2 rg_nseg2

CANFD 仲裁域波特率参数 tseg2

Bit 1:0 rg_nseg1[7:6]

CANFD 仲裁域波特率参数 tseg1 第 6、7 位

5.9.9.1.11 rg_dbt

Absolute Address: 0x1f2420

Base Offset: 0x20

Size: 0x2

Reset value: 0x0000

15:10	9:0
rg_dseg1	rg_dbrp
rw	rw

版权 © 2025 深圳市汇顶科技股份有限公司

Bit 15:10 rg_dseg1

CANFD 数据域波特率参数 tseg1

Bit 9:0 rg_nsjw

CANFD 数据域波特率参数 brp

5.9.9.1.12 rg_sspp_dbtu

Absolute Address: 0x1f2422

Base Offset: 0x22

Size: 0x2

Reset value: 0x0000

15:8	7:4	3:0
	rg_dseg2	rg_dsjw
	rw	rw

Bit 15:8 reserved

Bit 7:4 rg_dseg2

CANFD 数据域波特率参数 tseg2

Bit 3:0 rg_dsjw

CANFD 数据域波特率参数 sjw

5.9.9.1.13 rg_txbsel_acfsel

Absolute Address: 0x1f2438

Base Offset: 0x38

Size: 0x2

Reset value: 0x0000

15:5	4:0
	rg_acfsel
	rw

Bit 15:5 reserved

Bit 4:0 rg_acfsel

过滤器组选择，范围 0~7

5.9.9.1.14 rg_acr1_0

Absolute Address: 0x1f2410

Base Offset: 0x10

Size: 0x2

Reset value: 0xc000

15:8	7:0
rg_acr1	rg_acr0
Rw	rw

Bit 15:8 rg_acr1

过滤器比较值寄存器 1

版权 © 2025 深圳市汇顶科技股份有限公司

Bit 7:0 rg_acr0

过滤器比较值寄存器 0

寄存器 rg_acr3_2 类似 4

5.9.9.1.15 rg_amr1_0

Absolute Address: 0x1f2414

Base Offset: 0x14

Size: 0x2

Reset value: 0x1f00

15:8	7:0
rg_amr1	rg_amr0
Rw	rw

Bit 15:8 rg_amr1

过滤器比较掩码寄存器 1，每位为 0 时进行过滤检查，为 1 时不关心

Bit 7:0 rg_amr0

过滤器比较掩码寄存器 0，每位为 0 时进行过滤检查，为 1 时不关心

寄存器 rg_amr3_2 类似

5.10 LIN

GE1101 支持 LIN 模式对外通讯，LIN 的配置和使用有独立的子块协议，Header 中的数据类型需要设置为 LIN，本章详细介绍 LIN 的子块协议、使用方法及寄存器。

5.10.1 写寄存器

支持在一个子块中写单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x0 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，写寄存器时固定为 0x0。

reg_addr_0：访问的寄存器绝对地址。

reg_data_0：写入的寄存器值。

5.10.2 读寄存器命令

支持在一个子块中读单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x1 4bit	reg_addr_0 32bit	reg_addr_1 32bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，读寄存器时固定为 0x1。

reg_addr_0：访问的寄存器绝对地址。

 版权 © 2025 深圳市汇顶科技股份有限公司

5.10.3 读寄存器值

在主机发送读寄存器命令后，GE1101 会返回相应的寄存器值，支持一个或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x2 4bit	reg_addr_0 32bit	reg_data_0 16bit	…

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

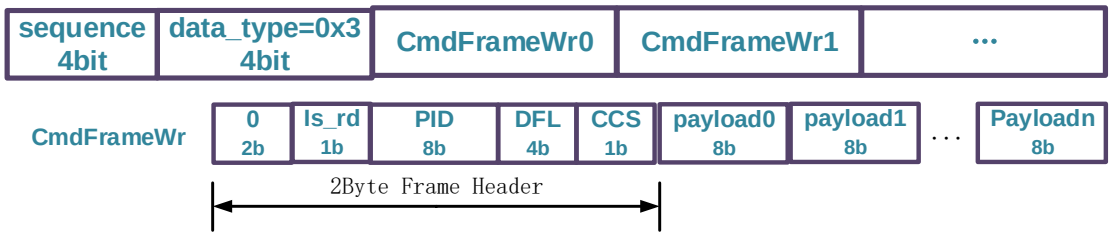
data_type：子块类型码，返回寄存器值时固定为 0x2。

reg_addr_0：读出的寄存器绝对地址。

reg_data_0：读出的寄存器值。

5.10.4 LIN 发送数据

支持在一次子块通讯中，对 LIN 总线发起一帧或多帧 LIN 数据传输。



sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，发送数据时固定为 0x3。

Cmdframewr：LIN 数据帧，其中前 2 字节是帧头，后面根据数据长度填充数据

Is_rd：标志读写，0：写，1：读

PID：帧 ID（低 6 位）及校验位（高 2 位），生成方法如下

```
uint8_t LIN_GetPID(uint8_t FrameID) {
    uint8_t P0 = ((FrameID & 0x01) ^ ((FrameID >> 1) & 0x01) ^ ((FrameID >> 2) & 0x01) ^
((FrameID >> 4) & 0x01)) << 6;
    uint8_t P1 = (~(((FrameID >> 1) & 0x01) ^ ((FrameID >> 3) & 0x01) ^ ((FrameID >> 4) &
0x01) ^ ((FrameID >> 5) & 0x01))) << 7;
    return FrameID | P0 | P1;
}
```

DFL：数据长度

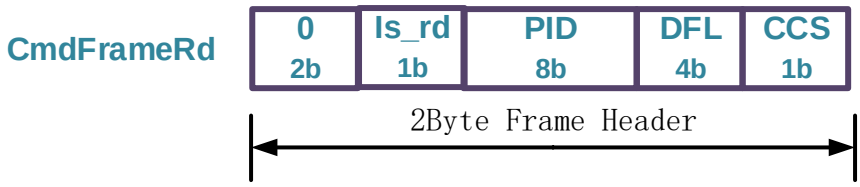
CCS：校验方式　0：经典校验　1：增强校验

Payload:数据

5.10.5 读取 LIN 数据命令

支持在一次子块通讯中，包含一次或多次 LIN 总线数据读操作。

sequence 4bit	data_type=0x4 4bit	CmdFrameRd0	CmdFrameRd1	…



sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，读取数据命令时固定为 0x4。

Cmdframewr：LIN 数据帧头

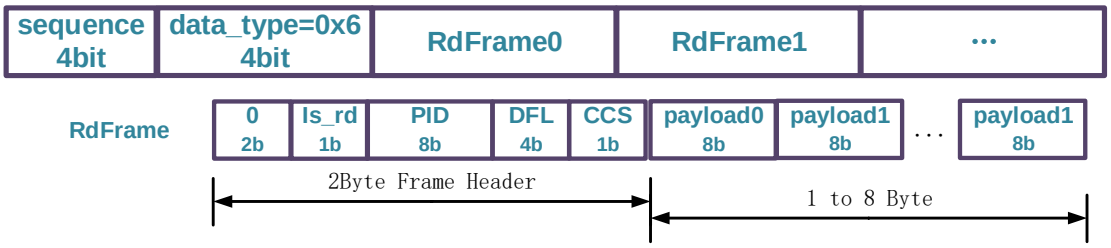
PID：帧 ID（低 6 位）及校验位（高 2 位），生成方法参考前述

DFL：数据长度

CCS：校验方式　0：经典校验　1：增强校验

5.10.6 读取 LIN 数据

当总线上有 LIN 数据回复时，GE1101 会返回 LIN 总线上的数据子块内容如下。



sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回总线数据时固定为 0x6。

Rdframe:返回的 LIN 数据帧，包含 2 字节帧头和数据

Is_rd：标志读写，0：写，1：读

PID：帧 ID（低 6 位）及校验位（高 2 位）

DFL：数据长度

CCS：校验方式　0：经典校验　1：增强校验

Payload:数据

5.10.7 过滤器

LIN 支持一组 ID 过滤器，支持 6bit 的 ID 过滤，修改 LIN 寄存器 rg_id_mask，设置需要关心的位，修改 LIN 寄存器 rg_id_filter 设置过滤的值。

5.10.8 常用流程

初始化：

1. 修改 LIN 寄存器 LIN0_CTRL，打开 LIN 功能
2. 修改 LIN 寄存器 LIN0_BRR，设置波特率

发送数据：

版权 © 2025 深圳市汇顶科技股份有限公司

1. 填充子块数据中的 CmdFrameWr，根据 ID 填充 PID，根据数据长度和内容，填充 DFL 和 payload，根据数据校验方式填充 CCS
2. 通过主机通讯接口将子块数据发给芯片

接收数据：

1. 填充子块数据中的 CmdFrameRd，根据 ID 填充 PID，根据校验方式填充 CCS，is_rd 填充为 1
2. 通过主机通讯接口将子块数据发给芯片
3. LIN 总线收到返回数据
4. 芯片通过主机通讯接口将子块数据发给主机
5. 解析 Header 和子块数据，解析 RdFrame，获取 PID、DFL、CCS 和 payload
6. 根据 PID 解析 ID，获取 payload

过滤器设置：

1. 修改 LIN 寄存器 LIN0_FILTER，设置过滤掩码和过滤值

5.10.9 参考实现

参考 SDK 中如下接口

uint8_t ge_api_lin_set_config(uint8_t node_index, ge_lin_config_t *config, uint16_t time_out_ms)

uint8_t ge_api_lin_write_sensor_data(uint8_t node_index, uint8_t frameID, uint8_t ccs, uint8_t *data, uint8_t len, uint16_t time_out_ms)

uint8_t ge_api_lin_read_sensor_data(uint8_t node_index, uint8_t frameID, uint8_t ccs, uint8_t *data, uint8_t len, uint16_t time_out_ms)

5.10.10 寄存器

5.10.10.1.1 LIN0_CTRL

Absolute Address: 0x1f2600

Base Offset: 0x0

Size: 0x2

Reset value: 0x0001

15	14	13:1	0
	rg_enable		rg_mme
	rw		rw

Bit 15 reserved

Bit 13:1 reserved

Bit 14 rg_enable

LIN 模块使能开关

0：关闭

1：打开

Bit 0 rg_mme

LIN 模块模式

1：master 模式（默认）

5.10.10.1.2 LIN0_BRR_H

Absolute Address: 0x1f2612

Base Offset: 0x12

Size: 0x2

Reset value: 0x00c8

15	14:0
	rg_dii
	Rw

Bit 15 reserved

Bit 14:0 rg_dii

波特率分频系数整数部分

波特率分频系数计算方式如下

Baud_div = fref / baud / 16，举例若参考时钟 fref 为 24.576M，目标波特率 baud 为 19.2K，计算结果波特率分频系数整数部分值为 80

5.10.10.1.3 LIN0_BRR_L

Absolute Address: 0x1f2610

Base Offset: 0x10

Size: 0x2

Reset value: 0x0000

15:4	3:0
	rg_dif
	Rw

Bit 15:4 reserved

Bit 3:0 rg_dif

波特率分频系数小数部分，步进为 0.0625

波特率分频系数计算方式如下

Baud_div = fref / baud / 16，举例若参考时钟 fref 为 24.576M，目标波特率 baud 为 19.2K，计算结果波特率分频系数整数部分值为 81，小数部分为 0.138，则此时设置 rg_dif 为 0x2 即可。

5.10.10.1.4 LIN0_FILTER

Absolute Address: 0x1f2630

Base Offset: 0x30

Size: 0x2

Reset value: 0x0000

15:12	11:6	5:0
	rg_id_mask	rg_id_filter
	rw	rw

Bit 15:12 reserved

Bit 11:6 rg_id_mask

接收 ID 过滤掩码，每一位对应接收的 ID 的每一位，该位为 1 是进行该位的过滤，否则不关系该位的值

Bit 5:0 rg_id_filter

接收 ID 过滤匹配值，每一位对应接收的 ID 的每一位，需要过滤时检查该 bit 上的值是否与接收的 ID 一致

5.11 UART

GE1101 支持 UART 模式对外通讯，最高速率 115200，UART 的配置和使用有独立的子块协议，Header 中的数据类型需要设置为 UART，本章详细介绍 UART 的子块协议、使用方法及寄存器。

5.11.1 写寄存器

支持在一个子块中写单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x0 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，写寄存器时固定为 0x0。

reg_addr_0：访问的寄存器绝对地址。

reg_data_0：写入的寄存器值。

5.11.2 读寄存器命令

支持在一个子块中读单个寄存器或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x1 4bit	reg_addr_0 32bit	reg_addr_1 32bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，读寄存器时固定为 0x1。

reg_addr_0：访问的寄存器绝对地址。

5.11.3 读寄存器值

在主机发送读寄存器命令后，GE1101 会返回相应的寄存器值，支持一个或多个寄存器，子块内容如下。

sequence 4bit	data_type=0x2 4bit	reg_addr_0 32bit	reg_data_0 16bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，返回寄存器值时固定为 0x2。

reg_addr_0：读出的寄存器绝对地址。

reg_data_0：读出的寄存器值。

5.11.4 UART 发送数据

Uart 发送数据子块协议如下

版权 © 2025 深圳市汇顶科技股份有限公司

sequence 4bit	data_type=0x3 4bit	wr_data_0 8bit	wr_data_1 8bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，发送数据时固定为 0x3。

wr_data_x：发送的第 x 个数据

5.11.5 读取 UART 数据

当 Uart 总线上收到数据后，会返回到主机通信接口中，数据子块协议如下

sequence 4bit	data_type=0x4 4bit	rd_data_0 8bit	rd_data_1 8bit	...

sequence: 序列号

范围：0x0~0xF，用于区分每个多个子块的顺序，使用时建议递增，到达 0xF 后回归为 0x0。

data_type：子块类型码，接收数据时固定为 0x4。

rd_data_x：接收的第 x 个数据

5.11.6 常用流程

初始化：

1. 修改 UART 寄存器 LCR_L，打开分频系数访问权限
2. 修改 UART 寄存器 RBR_L，根据目标波特率设置分频系数
3. 修改 UART 寄存器 LCR_L，关闭分频系数访问权限
4. 修改 UART 寄存器 LCR_L，设置数据位长度、校验位方式、停止位
5. 修改 UART 寄存器 SFE_L，使能 FIFO
6. 修改 UART 寄存器 RDPL，设置接收缓存长度为 0xFF
7. 修改 UART 寄存器 TimeOut_value，设置接收超时计时值

发送数据：

1. 根据要发送的数据，填充子块中的 wr_data
2. 通过主机通讯接口发送给芯片

接收数据：

1. UART 总线上收到数据
2. 芯片通过主机通讯即可发送给主机
3. 解析 Header、子块，获取 rd_data

5.11.7 参考实现

参考 SDK 中如下接口

uint8_t ge_api_uart_config(uint8_t node_index, uint32_t baud, uint16_t time_out_ms)

版权 © 2025 深圳市汇顶科技股份有限公司

uint8_t ge_api_uart_tx(uint8_t node_index, uint8_t *tx_data, uint16_t data_len, uint16_t time_out_ms)

uint8_t ge_api_uart_rx(uint8_t node_index, uint8_t *rx_data, uint16_t *data_len, uint16_t time_out_ms)

5.11.8 寄存器

5.11.8.1.1 LCR_L

Absolute Address: 0x1f2c0c

Base Offset: 0xc

Size: 0x2

Reset value: 0x0000

15:8	7	6:5	4	3	2	1:0
	DLAB		EPS	PEN	STOP	DLS
	rw		rw	rw	rw	rw

Bit 15:8 reserved

Bit 6:7 reserved

Bit 7 DLAB

分频系数访问开关

0：关闭

1：打开

Bit 4 EPS

奇偶校验选择

0：奇校验

1：偶校验

Bit 3 PEN

校验位使能

0：关闭

1：打开

Bit 2 STOP

停止位个数

0: 1

1: 1.5

Bit 1:0 DLS

数据长度

0：5bit

1：6bit

2：7bit

3：8bit

5.11.8.1.2 DLL

Absolute Address: 0x1f2c00

Base Offset: 0x0

Size: 0x2

Reset value: 0x0000

15:8	7:0
	DLL
	rw

Bit 15:8 reserved

Bit 7:0 DLL

分频系数低位

5.11.8.1.3 DLH

Absolute Address: 0x1f2c04

Base Offset: 0x4

Size: 0x2

Reset value: 0x0000

15:8	7:0
	DLH
	rw

Bit 15:8 reserved

Bit 7:0 DLH

分频系数高位

UART 波特率计算方式 baud = fref / 16 / div，举例：假设系统时钟为 24.576M，目标波特率为 9600，则分频系数 div = 160。

5.11.8.1.4 DLF

Absolute Address: 0x1f2cc0

Base Offset: 0xc0

Size: 0x2

Reset value: 0x0000

15:6	5:0
	DLF
	rw

Bit 15:6 reserved

Bit 5:0 DLF

分频系数小数部分，步进为 0.015625

UART 波特率计算方式 baud = fref / 16 / div，举例：假设系统时钟为 25M，目标波特率为 9600，则分频系数 div = 162.76，此时 DLL 为 162，DLF 设置为 48 或者 49 即可，设置时考虑波特率误差不超过 2%即可。

若设置 DLF 为 49，DLL 为 162，实际分频系数为 162.765，实际波特率为 9599.72，波特率误差为 0.0029%，符合误差范围。

5.11.8.1.5 SFE_L

Absolute Address: 0x1f2c98

Base Offset: 0x98

Size: 0x2

Reset value: 0x0000

15:1	0
	SFE
	rw

Bit 15:1 reserved

Bit 0 SFE

FIFO 使能开关

0：关闭

1：打开

5.11.8.1.6 RDPL

Absolute Address: 0x1f2d00

Base Offset: 0x0

Size: 0x2

Reset value: 0x0000

15:9	8:0
	RDPL
	rw

Bit 15:9 reserved

Bit 8:0 RDPL

接收缓存长度

5.11.8.1.7 TimeOut_value_L

Absolute Address: 0x1f2d0c

Base Offset: 0xc

Size: 0x2

Reset value: 0xffff

15:0
TimeOut_Value_L
rw

Bit 15:0 TimeOut_Value_L

接收超时时间低位

5.11.8.1.8 TimeOut_value_H

Absolute Address: 0x1f2d0e

Base Offset: 0xe

Size: 0x2

Reset value: 0x0002

版权 © 2025 深圳市汇顶科技股份有限公司

15:2	1:0
	TimeOut_Value_H
	rw

Bit 15:2 reserved

Bit 1:0 TimeOut_Value_H

接收超时时间高位

UART 接收到数据之后先放在缓存内部，一段时间内连续接收数据，当没有数据接收并达到 Timeout 时间之后，将缓存中的数据一次通过主机接口传输给主机，推荐 Timeout_Value 保持默认值。

5.12 PTP

5.12.1 PTP 模块介绍

PTP（Precision Time Protocol，精确时间协议）是一种基于 IEEE 1588 标准 的网络时钟同步协议，旨在通过局域网（LAN）实现设备间亚微秒级的时间同步精度。PTP 分为主节点和从节点。通过 master 端和 slave 端交互时间戳报文，动态计算网络延迟并调整时钟偏差，从而达到主从节点的时间同步。PTP 模块遵循的是 IEEE 1588 V2 协议。

5.12.2 PTP 模块特性

1. 支持多主多从时间同步
2. 100ns 的时间精度
3. 支持 1PPS
4. 实时锁定时间戳，动态获取
5. 中断触发
6. 支持修正域
7. 支持 25M 和 24.576M 两种时钟频率下的时间戳计数

5.12.3 PTP 模块配置

1. rg_ptp_enable == 1，使能 PTP 模块；
2. rg_ptp_count_clk_sel = 0，时间戳计数模式选择 25M 计数模式，
    i. rg_ptp_count_clk_sel = 1，时间戳计数模式选择 24.576M 计数模式；
3. rg_ptp_phy_sel = 0，时间同步 phy 端口选择 phya，
    i. rg_ptp_phy_sel = 1，时间同步 phy 端口选择 phyb；
4. rg_ptp_get_ts_mask = 0xF，时间戳报文使能；
5. rg_ptp_irq_mask = 0xF，中断使能；
6. rg_irq_func_sel = 1，1PPS 引脚输出选定（可选操作）；

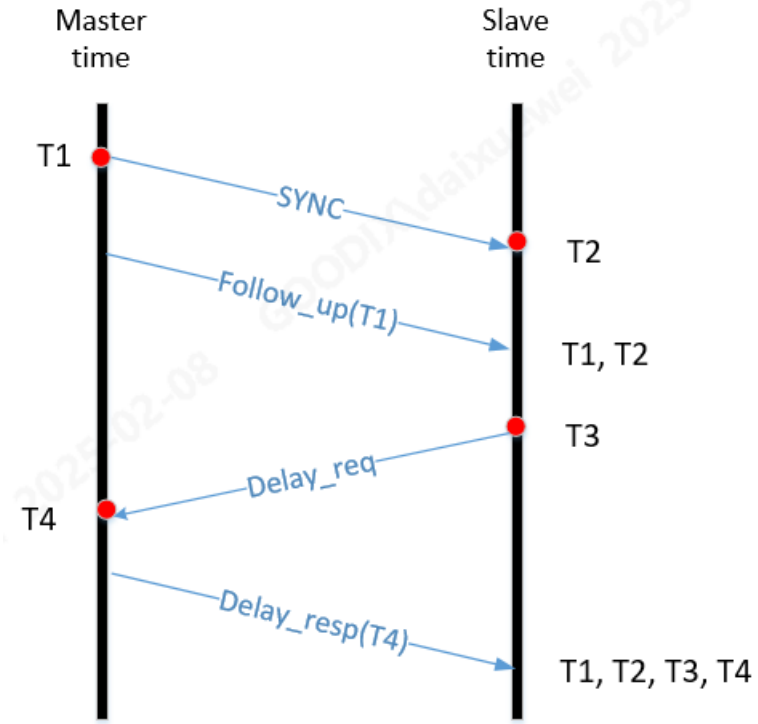
 版权 © 2025 深圳市汇顶科技股份有限公司

7. 主节点发送携带本地实时时间戳的 sync 报文到从节点，从节点收到第一个 sync 报文后更新本地时间戳。主节点和从节点进行初步的时间同步；

   - rg_ptp_rd_ts_command = 1，实时时间戳锁定
   - ptp_ts_latch_0~4，读取 80bit 的实时时间戳。高 48bit 以 s 为单位，低 32bit 以 ns 为单位，低 32bit 计数到 1s 向高 48bit 进位。

8. 主节点发送 sync 报文，并记录时间戳 T1;
9. ptp_tx_ts0~4，读取 80bit 的报文发送时间戳。
10. 主节点发送 follow_up 报文，follow_up 报文包含时间戳 T1;
11. 从节点接收 sync 报文，并记录时间戳 T2;
12. ptp_rx_ts0~4，读取 80bit 的报文接收时间戳。
13. 从节点发送 delay_req 报文，并记录时间戳 T3;
14. ptp_tx_ts0~4，读取 80bit 的报文发送时间戳。
15. 主节点接收 delay_req 报文，并记录时间戳 T4;
16. ptp_tx_ts0~4，读取 80bit 的报文发送时间戳。
17. 主节点发送 delay_resp 报文，delay_resp 报文包含时间戳 T4;
18. 从节点通过时间戳 T1，T2，T3，T4 计算主从节点时间偏移 offset=((T2-T1)-(T4-T3))/2;
19. 将时间偏移 offset（32bit）的低 16bit 写入寄存器 rg_ptp_offset_0，再将高 16bit 写入寄存器 rg_ptp_offset_1 触发从节点的时间戳更新。

版权 © 2025 深圳市汇顶科技股份有限公司



5.12.4 PTP 寄存器

5.12.4.1.1 local_reg_phy_ptp_rx_ts_n(n=0:4)

Absolute Address: 0x1f1600

Base Offset: n*0x2

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ptp_rx_ts_n															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

报文接收时间戳 80bit，其中高 48bit 以秒为单位，低 32bit 以纳秒为单位。ptp_rx_ts_0 为时间戳的最低 16bit，ptp_rx_ts_4 为时间戳的最高 16bit

5.12.4.1.2 local_reg_phy_ptp_tx_ts_n(n=0:4)

Absolute Address: 0x1f1600

Base Offset: 0xa+n*0x2

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ptp_tx_ts_n															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

报文发送时间戳 80bit，其中高 48bit 以秒为单位，低 32bit 以纳秒为单位。ptp_tx_ts_0 为时间戳的最低 16bit，ptp_tx_ts_4 为时间戳的最高 16bit。

5.12.4.1.3 local_reg_phy_ptp_ts_latch_n(n=0:4)

Absolute Address: 0x1f1600

版权 © 2025 深圳市汇顶科技股份有限公司

Base Offset: 0x14+n*0x2

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ptp_ts_latch_n															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

实时时间戳 80bit，其中高 48bit 以秒为单位，低 32bit 以纳秒为单位。ptp_tx_ts_0 为时间戳的最低 16bit，ptp_tx_ts_4 为时间戳的最高 16bit。

5.12.4.1.4 local_reg_phy_ptp_ctrl

Absolute Address: 0x1f1600

Base Offset: 0x26

Size: 0x2

Reset value: 0x00C0

15	14	13	12	11	10	9	8	7	6
-								rg_ptp_count_clk_sel	rg_ptp_custom_en
-								rw	rw

5	4	3	2	1	0
rg_ptp_phy_sel	rg_ptp_rd_ts_command	-	-	rg_ptp_corr_en	rg_ptp_enable
rw	rw	-	-	rw	rw

Bits 15:8 Reserved

Bit 7 **rg_ptp_count_clk_sel**

时间戳计数模式选择：

0：25M 计数模式

1：24.576M 计数模式

Bit 6 **rg_ptp_custom_en**

报文解析格式使能，使用默认值 1

Bit 5 **rg_ptp_phy_sel**

时间同步 phy 端口选择：

0: phya

1: phyb

Bit 4 **rg_ptp_rd_ts_command**

写 1，锁定实时时间戳

Bits 3:2 Reserved

Bit 1 **rg_ptp_corr_en**

写 1，使能修正域功能

Bit 0 **rg_ptp_enable**

写 1，使能 ptp 模块

5.12.4.1.5 local_reg_phy_ptp_etype

Absolute Address: 0x1f162c

Base Offset: 0x2c

Size: 0x2

Reset value: 0x88f7

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_ptp_etype															
rw															

Bits 15:0 **rg_ptp_etype**

PTP 报文以太网类型，默认为 0x88F7

5.12.4.1.6 local_reg_phy_ptp_offset_0

Absolute Address: 0x1f162e

Base Offset: 0x2e

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_ptp_offset_0															
rw															

Bits 15:0 **rg_ptp_offset_0**

设置 32bits 时间戳偏移值中的低 16bits

5.12.4.1.7 local_reg_phy_ptp_offset_1

Absolute Address: 0x1f1630

Base Offset: 0x30

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_ptp_offset_1															
rw															

Bits 15:0 **rg_ptp_offset_1**

设置 32bits 时间戳偏移值中的高 16bits

5.12.4.1.8 local_reg_phy_ptp_get_ts_mask

Absolute Address: 0x1f1632

Base Offset: 0x32

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-									rg_ptp_get_ts_mask						
-									rw						

Bits 15:7 Reserved

Bits 6:0 rg_ptp_get_ts_mask

报文时间戳使能掩码

<0>: sync 报文

<1>: follow_up 报文

<2>: delay_req 报文

<3>: delay_resp 报文

<4:5>: 保留

5.12.4.1.9 local_reg_phy_ptp_irq_mask

Absolute Address: 0x1f1634

Base Offset: 0x34

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-									rg_ptp_irq_mask						
-									rw						

Bits 15:7 Reserved

Bits 6:0 rg_ptp_irq_mask

中断使能掩码

<0>: sync 报文

<1>: follow_up 报文

<2>: delay_req 报文

<3>: delay_resp 报文

<4:5>: 保留

5.12.4.1.10 local_reg_phy_ptp_1pps_cnt

Absolute Address: 0x1f1636

Base Offset: 0x36

Size: 0x2

Reset value: 0xc350

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_ptp_1pps_cnt															
rw															

Bits 15:0 **rg_ptp_1pps_cnt**

1PPS 高电平持续时间。

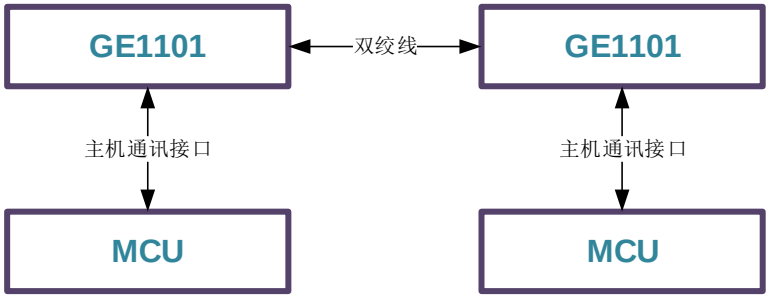
当 rg_ptp_count_clk_sel=0 时，时间单位为 40ns

当 rg_ptp_count_clk_sel=1 时，时间单位为 41ns

版权 © 2025 深圳市汇顶科技股份有限公司

6 MII/SPIS/I2CS（节点互传）

GE1101 支持多个带 MCU 的节点间互传数据，需要知道通讯双方的主机通讯接口以及网络位置（节点号、组网方式等）。



发送方：

1） Header 中填充目标节点号、数据类型（通讯接口类型），子块则没有协议要求，任何形式的数据都可以作为子块内容

2） 通过主机通讯接口下发给芯片

接收方：

1） 主机通过主机通讯接口收到芯片上报数据

2） 解析 Header 中目标节点号、数据类型（通讯接口类型），子块内容

通讯接口类型如下：

0x0：xMII

0x3：主机通讯接口 SPI Slave

0x5：主机通讯接口 I2C Slave

0x6：外设 CAN（FD）

6.1 INT（中断包）

中断包携带芯片的中断信息，以子块数据的形式传输给 MCU。主节点和连接 MCU 的子节点通过 xMII/SPI/I2C 发送中断包到 MCU。未连接 MCU 的子节点则将中断包放入超帧中，通过数据链路发送至主节点的 MCU 解析并做相应的处理。中断包长度为 192bits。常见中断信息有建链中断、断链中断、PTP 中断、睡眠唤醒中断和外设中断。

6.2 HEAR（维测包）

维护包提供了各外设接口和芯片数据控制层关于数据发送与接收的个数统计、数据控制层丢包统计、数据控制层剩余空间统计。通过周期性发送维测包，可实时体现网络的数据流量，同时也是定位问题的有效手段。维测包的长度为 512bits。

维护包字段	描述
maintain_info[511:496]	异常包丢包统计
maintain_info[495:480]	{11'd0，环逆时针/左菊花链标志，环顺时针/右菊花链标志_r，热节点标志}
maintain_info[479:464]	超时机制下，超时丢包统计
maintain_info[463:448]	数据控制层低优先级数据可写空间。单位：byte
maintain_info[447:432]	数据控制层缓存的低优先级数据包个数
maintain_info[431:416]	数据控制层高优先级数据可写空间。单位：byte

 版权 © 2025 深圳市汇顶科技股份有限公司

maintain_info[415:400]	数据控制层缓存的高优先级数据包个数
maintain_info[399:384]	数据控制层最高优先级数据可写空间。单位：byte
maintain_info[383:368]	数据控制层缓存的最高优先级数据包个数
maintain_info[367:352]	数据控制层写 mii 数据包个数
maintain_info[351:336]	数据控制层读 mii 数据包个数
maintain_info[335:320]	数据控制层读维护包的数据包个数
maintain_info[319:304]	数据控制层写 reg 的数据包个数
maintain_info[303:288]	数据控制层读 reg 的数据包个数
maintain_info[287:272]	数据控制层写 uart 的数据包个数
maintain_info[271:256]	数据控制层读 uart 的数据包个数
maintain_info[255:240]	数据控制层写 i2cm 的数据包个数
maintain_info[239:224]	数据控制层读 i2cm 的数据包个数
maintain_info[223:208]	数据控制层写 spim 的数据包个数
maintain_info[207:195]	数据控制层读 spim 的数据包个数
maintain_info[194:176]	数据控制层写 lin 的数据包个数
maintain_info[175:160]	数据控制层读 lin 的数据包个数
maintain_info[159:144]	数据控制层写 can 的数据包个数
maintain_info[143:128]	数据控制层读 can 的数据包个数
maintain_info[127:112]	数据控制层读中断包的数据包个数
maintain_info[111:96]	数据控制层写 PHY 的数据包个数
maintain_info[95:80]	数据控制层读 PHYA 的数据包个数
maintain_info[79:64]	数据控制层读 PHYB 的数据包个数
maintain_info[63:48]	数据控制层写 i2cs 的数据包个数
maintain_info[47:32]	数据控制层读 i2cs 的数据包个数
maintain_info[31:16]	数据控制层写 spis 的数据包个数
maintain_info[15:0]	数据控制层读 spis 的数据包个数

7 组网

7.1 组网介绍

GPAN 芯片遵循的是车载以太网协议，组网两端分别为 master 和 slave。将两个 GPAN 芯片分别设置为 master 和 slave，即可实现点对点组网。GPAN 芯片有 2 个网口可实现多个 GPAN 芯片进行组网，网络拓扑可为单菊花链，双菊花链和环形。

GPAN 至多支持 1 主子节点+61 子节点组网。

7.2 点对点

点对点组网是最简单的组网形式，主要用于端到端通信。

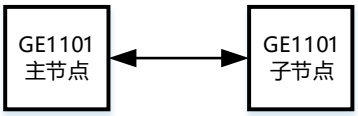


图 7-1 点对点组网

7.3 单菊花链

主节点只有一个网口进行组网，配合多个子节点，可组成单菊花链网络。

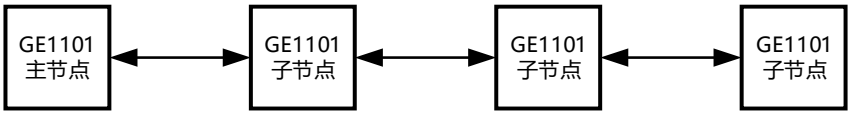


图 7-2 单菊花链组网

7.4 双菊花链

主节点两个网口均参与组网，配合多个子节点，可组成双菊花链网络。

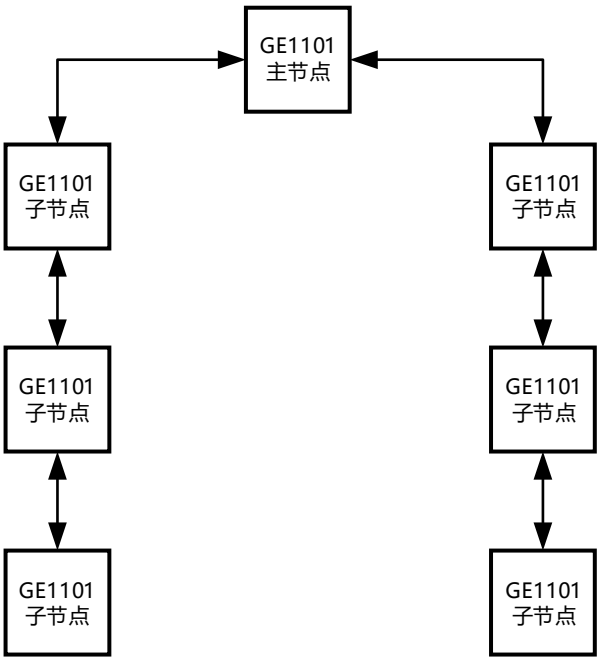


图 7-3 双菊花链组网

版权 © 2025 深圳市汇顶科技股份有限公司

7.5 环网

双菊花链的两个末节点相连，可组成环形网络。

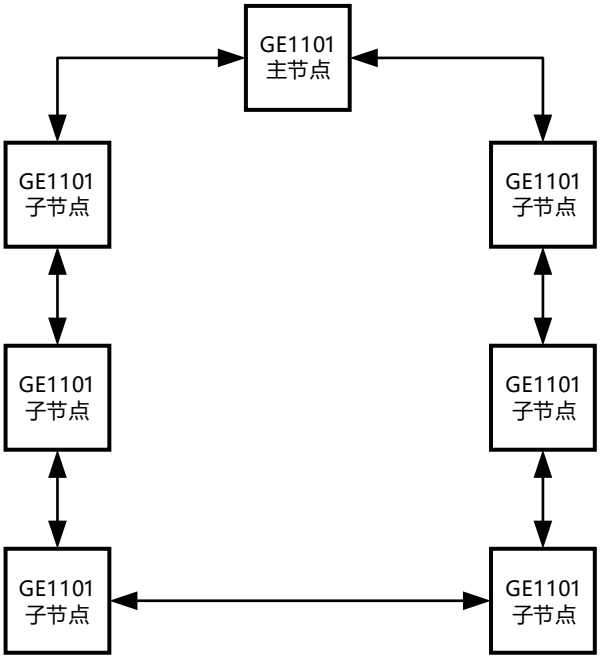
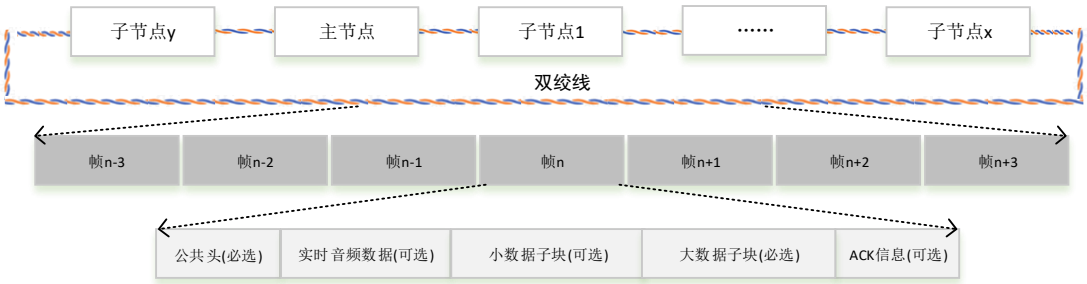


图 7-4 环形组网

7.6 数据帧

数据帧亦称超帧。数据帧携带各节点交互数据在网络中流动。数据帧由主节点和末节点发起。主节点和末节点是否有有效数据，都会发起数据帧。数据帧分为公共头、上下行音频数据、多个数据子块和多个 ACK。各字段定义可参考白皮书。



7.7 断网重连

7.7.1 菊花链断网重连

菊花链组网模式下，与主节点失去联系的子节点将进入孤岛模式。其他节点不受任何影响。比如图 5 中的子节点 3 进入孤岛模式，依赖于 GPAN 的处理器工作或者相连的外部 MCU 进行工作，但主节点、子节点 1 和子节点 2 节点之间的通信不受影响。断链状态将在 1ms 内上传到主节点。若子节点 2 和子节点 3 的网线恢复，将在 1ms 内上传建链成功状态到主节点。

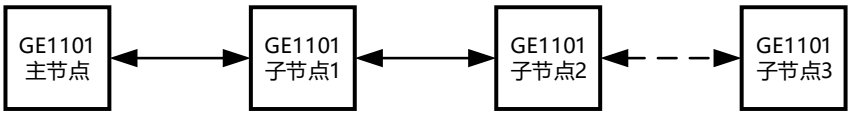


图 7-5 菊花链断网重连示意图

7.7.2 环网断网重连

在环模式下，如果出现节点故障或线束故障，需要快速的切换到菊花链，在 1ms 内上报告警并快速自

恢复，子节点自恢复流程如下：

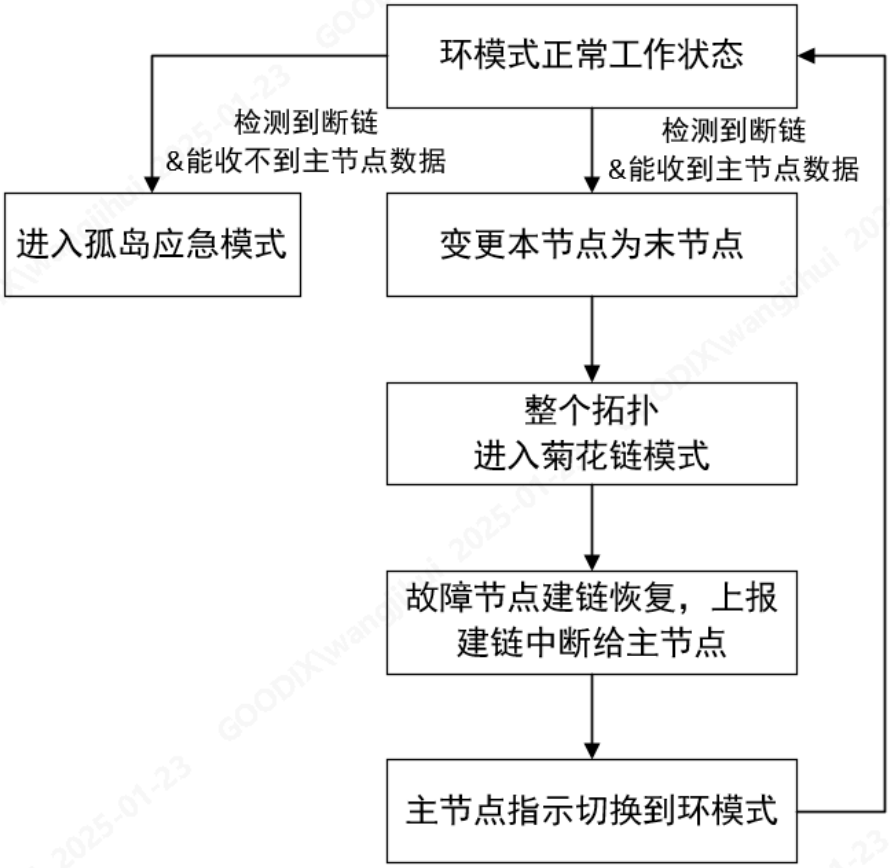


图 7-6 菊花链断网重连示意图

断网后子节点如果收到主节点消息，自动切换为末节点，并上传断网状态到主节点。

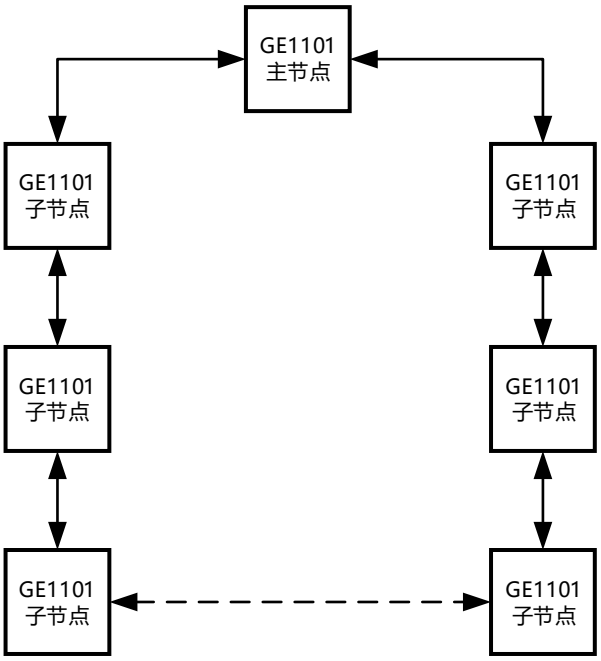


图 7-7 环形组网断网切换成双菊花链组网

断网后子节点不能收到主节点消息，子节点将进入孤岛模式。如图 8 子节点 X 进入孤岛模式。

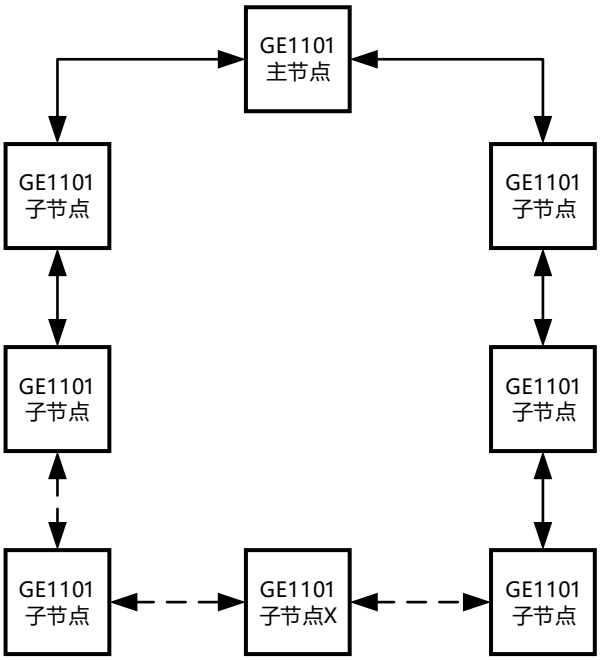


图 7-8 环形组网切换成双菊花链组网

7.8 组网配置流程

GPAN 简单的配置两个 PHY 端口的 master 和 slave 模式（默认是 slave 模式），即可实现点对点、单菊花链、双菊花链和环形组网。以下是 1 个主节点和 3 个子节点的组网流程，其他组网流程可类推。

1) rg_host_node_flag=1 配置主节点标志；

2) rg_this_node_num=0 配置主节点节点号为 0；

3) rg_phy_master_mode=1（phya_addrmap）配置主节点 PHYA 为 master，与子节点 1 的 PHYA 建链；

4) rg_this_node_num=1 配置子节点 1 节点号为 1；

5) rg_frame_loss_timer_enable=1 使能子节点 1 检测到断网后，自动发起上行包；

6) rg_phy_master_mode=1（phyb_addrmap）配置子节点 1 PHYB 为 master，与子节点 2 的 PHYA 建链；

7) rg_this_node_num=2 配置子节点 1 节点号为 2；

8) rg_frame_loss_timer_enable=1 使能子节点 2 检测到断网后，自动发起上行包；

9) rg_phy_master_mode=1（phyb_addrmap）配置子节点 2 PHYB 为 master，与子节点 3 的 PHYA 建链；

10) rg_this_node_num=3 配置子节点 3 节点号为 3；

11) rg_frame_loss_timer_enable=1 使能子节点 3 检测到断网后，自动发起上行包；

12) rg_phy_master_mode=1（phyb_addrmap）配置子节点 3 PHYB 为 master，与主节点的 PHYB 建链；

13) rg_daisy_chain_or_loop=1，配置主节点寄存器，标记网络拓扑为环形组网

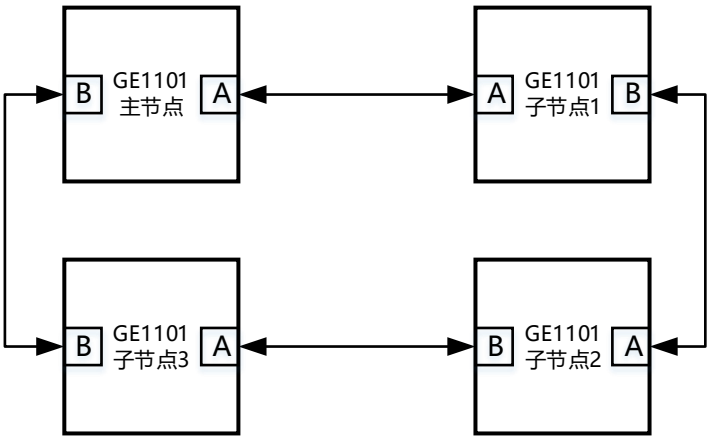


图 7-9 组网示意图

7.9 组网寄存器

7.9.1 local_reg_bus_node_config

Absolute Address: 0x1f1100

Base Offset: 0x0

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10
rg_phya_phyb_flag	rg_daisy_chain_or_loop	rg_frame_loss_timer_enable	-		rg_end_node_flag
rw	rw	rw	-		rw

9	8	7	6	5	4	3	2	1	0
-	rg_host_node_flag	-		rg_this_node_num					
-	rw	-		rw					

Bit 15 **rg_phya_phyb_flag**

0: 主节点 phya 发出的包标识为顺时针方向或右菊花链方向，phyb 发出的包标识为逆时针方向或左菊花链方向

1：主节点 phya 发出的包标识为逆时针方向或左菊花链方向，主节点 phyb 发出的包标识为顺时针方向或右菊花链方向

Bit 14 **rg_daisy_chain_or_loop**

0: 菊花链组网拓扑

1：环形组网拓扑

Bit 13 **rg_frame_loss_timer_enable**

1：使能子节点断网后自动发起上行包

0：子节点断网后不会发起上行包，需手动配置 rg_end_node_flag 产生上行包

Bits 12:11 Reserved

Bit 10 **rg_end_node_flag**

配置子节点为末节点

Bit 9 Reserved

Bit 8 **rg_host_node_flag**

配置节点为主节点，主节点将自动发起起始数据帧

Bits 7:6 Reseved

Bits 5:0 **rg_this_node_num**

节点号，主节点和各子节点的唯一标识。节点号最大值为 0x3e。

7.9.2 local_reg_bus_frame_period

Absolute Address: 0x1f1108

Base Offset: 0x8

Size: 0x2

Reset value: 0x0200

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_frame_period															
rw															

Bits 15:0 **rg_frame_period**

帧周期，主节点和末子节点发起起始帧的周期。有音频业务时，以音频周期为准。比如 48k 的音频业务在 24.576M 系统时钟下为 512 个系统时钟，帧周期为 512。

7.9.3 local_reg_bus_big_subframe_r_config

Absolute Address: 0x1f110a

Base Offset: 0xa

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	rg_big_subframe_data_r_len											-	rg_big_subframe_data_r_num		
-	rw											-	rw		

Bit 15 Reserved

Bits 14:4 **rg_big_subframe_data_r_len**

右菊花链或顺时针方向，大数据子块长度。单位为 byte

Bit 3 Reserved

Bits 2:0 **rg_big_subframe_data_r_num**

右菊花链或顺时针方向，大数据子块个数

7.9.4 local_reg_bus_little_subframe_r_config

Absolute Address: 0x1f110c

Base Offset: 0xa

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-					rg_little_subframe_data_r_len							-	rg_little_subframe_data_r_num		
-					rw							-	rw		

Bit 15:11 Reserved

Bits 10:4 **rg_little_subframe_data_r_len**

右菊花链或顺时针方向，小数据子块长度。单位为 byte

Bit 3 Reserved

Bits 2:0 **rg_little_subframe_data_r_num**

右菊花链或顺时针方向，小数据子块个数

7.9.5 local_reg_bus_big_subframe_l_config

Absolute Address: 0x1f110e

Base Offset: 0xe

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	rg_big_subframe_data_l_len											-	rg_big_subframe_data_l_num		
-	rw											-	rw		

Bit 15 Reserved

Bits 14:4 **rg_big_subframe_data_l_len**

左菊花链或逆时针方向，大数据子块长度。单位为 byte

Bit 3 Reserved

Bits 2:0 **rg_big_subframe_data_l_num**

左菊花链或逆时针方向，大数据子块个数

7.9.6 local_reg_bus_little_subframe_l_config

Absolute Address: 0x1f1110

Base Offset: 0x10

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-					rg_little_subframe_data_l_len							-	rg_little_subframe_data_l_num		
-					rw							-	rw		

Bit 15:11 Reserved

Bits 10:4 **rg_little_subframe_data_l_len**

左菊花链或逆时针方向，小数据子块长度。单位为 byte

Bit 3 Reserved

Bits 2:0 **rg_little_subframe_data_l_num**

左菊花链或逆时针方向，小数据子块个数

7.9.7 phy_config(phya_addrmap)

Absolute Address: 0x1e

Base Offset: 0x1e

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-															rg_phy_master_mode
-															rw

Bits 15:1 Reserved

Bit 0 **rg_phy_master_mode**

PHY 的 master 和 slave 模式配置

0：salve

1：maser

7.9.8 phy_config(phyb_addrmap)

Absolute Address: 0x8001e

Base Offset: 0x8001e

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-															rg_phy_master_mode
-															rw

Bits 15:1 Reserved

Bit 0 **rg_phy_master_mode**

PHY 的 master 和 slave 模式配置

0：salve

1：maser

7.9.9 local_reg_phy_link_state(local_phya_addrmap)

Absolute Address: 0x1f0002

Base Offset: 0x2

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	cqi_rank			rem_tx_mode_ok	loc_rcvr_status	tx_mode		-	sm_phy_ctrl			sm_ctrl			
-	r			r	r	r		-	r			r			

Bit 15 Reserved

Bits 14:12 **cqi_rank**

建链信号质量查询（local_reg_phy_link_state & 0xFFF=0xe48 时，值有效）

000：worse than class A SQI(unstable lint)
001：class A SQI(ubstable link)
010：class B SQI(ubstable link)
011：class C SQI(good link)

100：class D SQI(good link,bit error rate <1e-10)
101：class E SQI(good link)
110：class F SQI(very good link)
111：class G SQI(very good link)

Bit 11 **rem_tx_mode_ok**

远端建链状态

1：远端 link 正常

0：远端 link 异常

Bit 10 **loc_rcvr_status**

本地接收状态

1：正常

0：异常

Bits 9:8 **tx_mode**

0: SEND_Z; 1: SEND_I; 2: SEND_N

Bit 7 Reserved

Bits 6:4 **sm_phy_ctrl**

Phy_ctrl 状态机查询

000：CTL_DIS_TX
001：CTL_SLAVE_SILENT
010：CTL_TRAINING
011：CTL_SEND_IDLE
100：CTL_SEND_IDLE_DATA
101：CTL_SLEEP_SILENT
110：CTL_WAKE_TRANSMIT

Bits 3:0 **sm_ctrl**

Sm ctrl 状态机查询，正常值为 8

7.9.10 local_reg_phy_link_cnt(local_phya_addrmap)

Absolute Address: 0x1f0046

Base Offset: 0x46

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ro_phy_link_abnormal_cnt						ro_link_failure_cnt									
r						r									

Bits 15: 10 **ro_phy_link_abnormal_cnt**

断链次数统计

Bits 9:0 **ro_link_failure_cnt**

误码次数统计

7.9.11 local_reg_phy_link_state(local_phyb_addrmap)

Absolute Address: 0x1f0802

Base Offset: 0x2

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	cqi_rank			rem_tx_mode_ok	loc_rcvr_status	tx_mode		-	sm_phy_ctrl			sm_ctrl			
-	r			r	r	r		-	r			r			

Bit 15 Reserved

Bits 14:12 **cqi_rank**

建链信号质量查询（local_reg_phy_link_state & 0xFFF=0xe48 时，值有效）

000：worse than class A SQI(unstable lint)
001：class A SQI(ubstable link)
010：class B SQI(ubstable link)
011：class C SQI(good link)
100：class D SQI(good link,bit error rate <1e-10)
101：class E SQI(good link)
110：class F SQI(very good link)
111：class G SQI(very good link)

Bit 11 **rem_tx_mode_ok**

远端建链状态

1：远端 link 正常

0：远端 link 异常

Bit 10 **loc_rcvr_status**

本地接收状态

1：正常

0：异常

Bits 9:8 **tx_mode**

0: SEND_Z; 1: SEND_I; 2: SEND_N

Bit 7 Reserved

Bits 6:4 **sm_phy_ctrl**

Phy_ctrl 状态机查询

000：CTL_DIS_TX
001：CTL_SLAVE_SILENT
010：CTL_TRAINING
011：CTL_SEND_IDLE
100：CTL_SEND_IDLE_DATA
101：CTL_SLEEP_SILENT
110：CTL_WAKE_TRANSMIT

Bits 3:0 **sm_ctrl**

Sm ctrl 状态机查询，正常值为 8

7.9.12 local_reg_phy_link_cnt(local_phyb_addrmap)

Absolute Address: 0x1f0846

Base Offset: 0x46

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ro_phy_link_abnormal_cnt						ro_link_failure_cnt									
r						r									

Bits 15: 10 **ro_phy_link_abnormal_cnt**

断链次数统计

Bits 9:0 **ro_link_failure_cnt**

误码次数统计

8 音频

8.1 音频接口模块介绍

音频接口模块由于其灵活性和广泛的配置选项，可以支持一整套广泛的音频协议。包括 I2S 标准、I2S 左对齐、I2S 右对齐、TDM 模式 A、TDM 模式 B、PCMA-N、PCMB-N 和 Intel 25-FS Mode 等协议。

音频接口模块支持主、从两种工作模式。当工作在主模式时，位时钟（BCLK）和帧同步信号（SYNC）由芯片内部产生并送到输出端给从设备。当工作在从模式时，位时钟（BCLK）和帧同步信号（SYNC）来自于输入端。

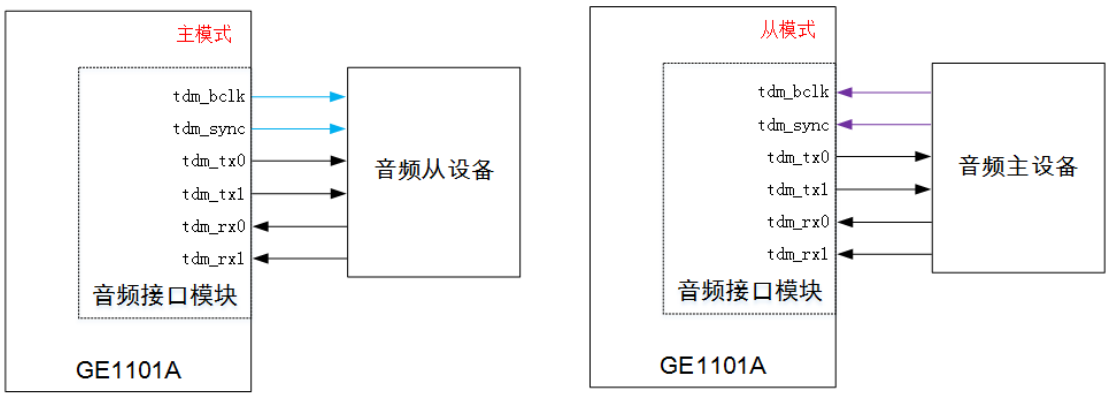


图 8-1 主模式和从模式

8.2 音频接口模块主要特性

1. 支持主、从两种工作模式。
2. 支持同时发送和接收音频数据。
3. 支持单、双线模式和交织模式。
4. 支持回环模式。
5. 主模式时通过时钟分频器实现 32KHz~192KHz 多种音频采样频率。
6. 支持 I2S 标准、I2S 左对齐、I2S 右对齐、TDM 模式 A、TDM 模式 B、PCMA-N、PCMB-N 和 Intel 25-FS Mode 等协议。
7. 数据大小可配置：8、16、20、24、32 位。
8. 通道宽度可配置：16、20、24、32 位。
9. 在 TDM 模式下支持多达 32 个可配置大小的通道。
10. 支持音频输入数据和输入缓存之间的重影射。

8.3 音频时钟生成

在主模式下音频位时钟（BCLK）和帧同步信号（SYNC）由芯片内部产生，如图 x 所示。

版权 © 2025 深圳市汇顶科技股份有限公司

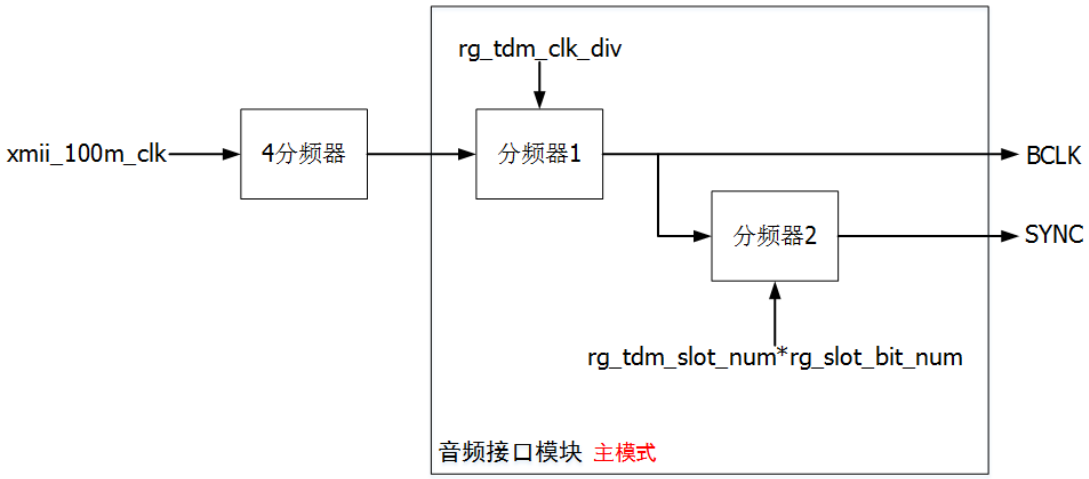


图 8-2 音频时钟生成

xmii_100m_clk 来自于芯片内部 PHY 输出，在音频应用场景下等于 24.576MHz*4=98.304MHz。通过一个固定 4 分频器后输入给音频接口模块。用户使用 rg_tdm_clk_div、rg_tdm_slot_num 和 rg_slot_bit_num 寄存器进行配置来适配不同音频应用场景。

表 3. 音频时钟常用配置

Input clock(MHz)	rg_tdm_clk_div	rg_tdm_slot_num	rg_slot_bit_num	BCLK(MHz)	SYNC(KHz)
24.576	1/24	2	16	1.024	32
	1/12	2	32	2.048	32
	1/3	8	32	8.192	32
	1/16	2	16	1.536	48
	1/8	2	32	3.072	48
	1/2	8	32	12.288	48
	1/8	2	16	3.072	96
	1/4	2	32	6.144	96
	1/1	8	32	24.576	96
	1/4	2	16	6.144	192
	1/8	2	32	12.288	192
	1/2	4	32	24.576	192

8.4 音频数据帧

音频数据帧是超帧的一部分，用户可以根据需求在超帧中开辟一段固定的数据空间作为音频数据帧。各节点可以按照配置的格式分别存入或取出音频数据。

图 8-3 音频数据帧格式

主要特性:

1. 音频数据帧支持多达 32 个通道的音频数据。
2. 每个音频通道支持最高 32 位宽度的音频数据。
3. 每个音频通道支持 4 位 CRC 校验。

4. 每个节点的音频接口模块可以通过灵活配置的通道映射关系表（audio_a2b_tdm_putx 或 audio_b2a_tdm_putx），将接收的音频数据存入音频数据帧的各通道中。
5. 每个节点的音频接口模块可以通过灵活配置的通道映射关系表（audio_a2b_tdm_get_bitmap_l/h 或 audio_b2a_tdm_get_bitmap_l/h），将音频数据帧各通道的数据取出并发送出去。

8.5 音频配置流程

1. local_reg_tdm_mst_slv[0] = 1，主模式；local_reg_tdm_mst_slv[0] = 0，从模式。
2. local_reg_tdm_channel_en [0] = 1，使能输出通道； local_reg_tdm_channel_en [1] = 1，使能输入通道。
3. 根据实际音频格式配置 local_reg_tdm_config， 例如 0x4433: TDM_B, 音频数据宽度=32 位, 音频通道宽度=32 位, 8 通道。
4. 配置 local_reg_tdm_clk_div 生成音频时钟。
5. 配置 rg_tdm_pin_num = 0 单线模式，rg_tdm_pin_num = 1 双线模式。
6. 配置 rg_tdm_interleave = 1 交织使能。
7. 配置 local_reg_bus_audio_r_len 右链/顺时针音频数据帧长度。
8. 配置 local_reg_bus_audio_l_len 左链/逆时针音频数据帧长度。
9. 配置 local_reg_bus_frame_period 帧周期。
10. 配置 audio_tdm_bit_len 音频数据帧音频通道宽度。
11. 配置 audio_a2b_tdm_get_bitmap_l，audio_a2b_tdm_get_bitmap_h，audio_b2a_tdm_get_bitmap_l 和 audio_b2a_tdm_get_bitmap_h 获取音频数据帧中音频通道的映射表。
12. 配置 audio_a2b_tdm_put0~15 和 audio_b2a_tdm_put0~15 存入音频数据帧的影射表。
13. local_reg_tdm_enable[0] = 1，释放复位信号使能音频接口模块。

8.6 音频寄存器

8.6.1 local_reg_tdm_enable

Absolute Address: 0x1f1000

Base Offset: 0x0

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
															rg_rst_n
															rw

Bits 15:1 Reserved

Bit 0 **rg_rst_n**

I2S/TDM reset 控制。

0：I2S/TDM 处于 Reset 状态（关闭）

1：I2S/TDM 从 Reset 状态释放（打开）

8.6.2 local_reg_tdm_mst_slv

Absolute Address: 0x1f1002

Base Offset: 0x2

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
															rg_mst_slv
															rw

Bits 15:1 Reserved

Bit 0 **rg_mst_slv**

位时钟/帧同步信号的主从控制。

0：从模式，即位时钟/帧同步信号来自于输入端

1：主模式，即位时钟/帧同步信号内部产生，并送到输出端给从设备

8.6.3 local_reg_tdm_channel_en

Absolute Address: 0x1f1004

Base Offset: 0x4

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
													rg_fspw_sel	rg_in_en	rg_out_en
													rw	rw	rw

Bits 15:3 Reserved

Bit 2 **rg_fspw_sel**

帧同步信号的脉冲宽度。

0：帧同步脉冲宽度为 1 个位时钟周期

1：帧同步脉冲宽度为 1 个 Slot 宽度的时钟周期

说明：只在 i2s_tdm_frmt_sel 为 TDM/PCM mode A 或 TDM/PCM mode B 的时候有效。

Bit 1 **rg_in_en**

输入通路使能。

0：输入通路关闭

1：输入通路使能

说明：配置为 0 时，I2S RX 通路不工作。该通路单独关掉不影响 TX 通路正常工作。

Bit 0 **rg_out_en**

输出通路使能。

0：输出通路关闭

1：输出通路使能

说明：配置为 0 时，I2S TX 通路不工作。该通路单独关掉不影响 RX 通路正常工作。

8.6.4 local_reg_tdm_config

Absolute Address: 0x1f1006

Base Offset: 0x6

Size: 0x2

Reset value: 0x3434

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	rg_frmt_sel				rg_sample_bit_num					rg_slot_bit_num			rg_tdm_slot_num		
	rw	rw	rw		rw	rw	rw			rw	rw		rw	rw	rw

Bit 15 Reserved

Bits 14:12 **rg_frmt_sel**

数据格式控制。

000b – I2S Normal (default)

001b – I2S Left-justified

010b - I2S Right-justified

011b – TDM/PCM mode A

100b – TDM/PCM mode B

101b – PCMA-N, Reserved

110b – PCMB-N, Reserved

111b – Intel 25-FS Mode, Reserved

Bit 11 Reserved

Bits 10:8 **rg_sample_bit_num**

控制样点的位宽。

000b – 8b

001b – 16b

010b – 20b

011b – 24b (default)

100b – 32b

101b~111b – Reserved

说明：控制 slot 中样点的有效位宽，必须小于或等于 slot 的位宽。

Bits 7:6 Reserved

Bits 5:4 **rg_slot_bit_num**

控制 slot 的位宽。

00b – 16b

01b – 20b

10b – 24b

11b – 32b (default)

说明：slot 位宽中大于样点位宽的低位比特补零。

Bit 3 Reserved

Bits 2:0 **rg_tdm_slot_num**

TDM 模式通道数，即每一帧的 slot 个数。

000b – 1-ch

001b – 2-ch (default)

010b – 4-ch

011b – 8-ch

100b -- 16-ch

101b -- 32-ch

others –2-ch

说明：数据格式选择 I2S 时，该配置无效，固定为 2 通道。选择 TDM 时，可以配置为 2,4,6,8 通道

8.6.5 local_reg_tdm_clk_div

Absolute Address: 0x1f1008

Base Offset: 0x8

Size: 0x2

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
								rg_tdm_clk_div							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:8 Reserved

Bits 7:0 **rg_tdm_clk_div**

master 模式下的时钟分频，对 24.576M 进行分频，支持整数分频。

0: 不分频

1: 二分频

2: 3 分频

……

n: n+1 分频

8.6.6 local_reg_tdm_enable

Absolute Address: 0x1f100a

Base Offset: 0xa

Size: 0x2

Reset value: 0x0200

15	14	13	12	11	10	9	8
	rg_pdm_start_addr				rg_tdm_pin_sel	rg_tdm_interleave	rg_tdm_pin_num
	rw	rw	rw		rw	rw	rw

7	6	5	4	3	2	1	0
rg_tx_test_en	rg_lpbk_en	rg_tx_bclk_inv	rg_tx_lrck_inv	rg_dout_at_pos_neg	rg_din_at_neg_pos	rg_rx_bclk_inv	rg_rx_lrck_inv
rw	rw	rw	rw	rw	rw	rw	rw

Bit 15 Reserved

Bits 14:12 **rg_pdm_start_addr**

pdm 有效的数据个数，0 为无 0: 不分频

Bit 11 Reserved

Bit 10 **rg_tdm_pin_sel**

单线模式下 pin 选择

0：rx0/tx0

1：rx1/tx1

Bit 9 **rg_tdm_interleave**

交织使能，置位为交织

Bit 8 **rg_tdm_pin_num**

单线模式和双线模式，置位为双线模式

Bit 7 **rg_tx_test_en**

测试模式使能，打开 tx 发送 rg_idle_data

Bit 6 **rg_lpbk_en**

I2S/TDM 环回控制。

0：正常模式，TX 与 RX 独立工作

1：环回使能，TX 数据传输到 RX

Bit 5 **rg_tx_bclk_inv**

I2S TX 位时钟极性选择。

0：正常极性

1：极性反转

说明：当配置为 1 时，TX 的输入 bclk 极性翻转，RX 不受影响。此寄存器为测试用。

Bit 4 **rg_tx_lrck_inv**

I2S TX 帧同步时钟极性选择。

0：正常极性

1：极性反转

说明：当配置为 1 时，TX 的输入 wclk 极性翻转，RX 不受影响。此寄存器为测试用。

Bit 3 **rg_dout_at_pos_neg**

输出数据发送沿控制。

0：TX 数据在下降沿发送（默认值）

1：TX 数据在上升沿发送

说明：I2S/TDM 的 TX 均是在下降沿发送数据。也可以配置为 1，在延后半拍的上升沿发送数据。根据外接第三方 IP 的 RX 端采样情况灵活配置。

Bit 2 **rg_din_at_neg_pos**

输入数据采样锁存控制。

0：RX 数据在上升沿采样（默认值）

1：RX 数据在下降沿采样

说明：I2S/TDM 的 RX 均是在上升沿采样数据。也可以配置为 1，在延后半拍的下降沿采样数据。根据外接第三方 IP 的 TX 端数据发送情况灵活配置。

Bit 1 **rg_rx_bclk_inv**

I2S RX 位时钟极性选择。

0：正常极性

1：极性反转

说明：当配置为 1 时，RX 的输入 bclk 极性翻转，TX 不受影响。此寄存器为测试用。

Bit 0 **rg_rx_lrck_inv**

I2S RX 帧同步时钟极性选择。

0：正常极性

1：极性反转

说明：当配置为 1 时，RX 的输入 wclk 极性翻转，TX 不受影响。此寄存器为测试用。

8.6.7 local_reg_bus_audio_r_len

Absolute Address: 0x1f1104

Base Offset: 0x4

Size: 0x2

Reset value: 0x0070

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
						rg_audio_r_len									
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:10 Reserved

Bits 9:0 **rg_audio_r_len**

tx 音频长度，单位为 4bit，右链(顺时针)

8.6.8 local_reg_bus_audio_l_len

Absolute Address: 0x1f1106

Base Offset: 0x6

Size: 0x2

Reset value: 0x0070

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
						rg_audio_l_len									
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:10 Reserved

Bits 9:0 **rg_audio_l_len**

tx 音频长度，单位为 4bit，左链(逆时针)

8.6.9 local_reg_bus_frame_period

Absolute Address: 0x1f1108

Base Offset: 0x8

Size: 0x2

Reset value: 0x0200

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
local_reg_bus_frame_period															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **local_reg_bus_frame_period**，帧周期

8.6.10 audio_tdm_bit_len

Absolute Address: 0x1f111c

Base Offset: 0x1c

Size: 0x2

Reset value: 0x0002

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
														rg_tdm_bit_len	
														rw	rw

Bits 15:2 Reserved

Bits 1:0 **rg_tdm_bit_len**

下行音频 控制 slot 的位宽

00: 16bit

01: 20bit

10: 24bit

11: 32bit

8.6.11 audio_a2b_tdm_get_bitmap_l

Absolute Address: 0x1f111e

Base Offset: 0x1e

Size: 0x2

Reset value: 0xffff

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_a2b_tdm_get_bitmap_l															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **rg_a2b_tdm_get_bitmap_l**，从 phya 进来的帧取音频的 bitmap

8.6.12 audio_a2b_tdm_get_bitmap_h

Absolute Address: 0x1f1120

Base Offset: 0x20

Size: 0x2

Reset value: 0xffff

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_a2b_tdm_get_bitmap_h															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **rg_a2b_tdm_get_bitmap_h**，从 phya 进来的帧取音频的 bitmap

8.6.13 audio_b2a_tdm_get_bitmap_l

Absolute Address: 0x1f1122

Base Offset: 0x22

Size: 0x2

Reset value: 0xffff

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_b2a_tdm_get_bitmap_l															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **rg_b2a_tdm_get_bitmap_l**，从 phyb 进来的帧取音频的 bitmap

8.6.14 audio_b2a_tdm_get_bitmap_h

Absolute Address: 0x1f1124

Base Offset: 0x24

Size: 0x2

Reset value: 0xffff

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rg_b2a_tdm_get_bitmap_h															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **rg_b2a_tdm_get_bitmap_h**，从 phyb 进来的帧取音频的 bitmap

8.6.15 audio_a2b_tdm_putn(n=0:15)

Absolute Address: 0x1f1128+n*0x2

Base Offset: 0x28+n*0x2

Size: 0x2

Reset value: 0x3f3f

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		rg_a2b_tdm_put_index(1+n*0x2)								rg_a2b_tdm_put_index(0+n*0x2)					
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

从 phyb 发出帧的音频通道上放入的 tdm rx 哪个通道，大于 31 表示不放入

8.6.16 audio_b2a_tdm_putn(n=0:15)

Absolute Address: 0x1f1148+n*0x2

Base Offset: 0x48+n*0x2

Size: 0x2

Reset value: 0x3f3f

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		rg_b2a_tdm_put_index(1+n*0x2)								rg_b2a_tdm_put_index(0+n*0x2)					
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

从 phya 发出帧的音频通道上放入的 tdm rx 哪个通道，大于 31 表示不放入

9 监听节点

网络监听是一种监视网络状态和网络数据传输的有效方法，监听节点是负责网络监听的特殊节点，用户可以使用监听节点监听和维护网络。

监听节点对于整个网络来说是透明的，即监听节点不影响网络的正常工作，网络中其它所有节点都无法访问监听节点。

监听节点的分布和网络中数据传输的机制相关。如图 4 所示的环状网络，数据帧主要有以下几种情况：

1) 主节点从顺时针方向发出起始帧，下游各子节点拿/存数据，环回到主节点结束。

2) 主节点从逆时针方向发出起始帧，下游各子节点拿/存数据，环回到主节点结束。

3) 在顺时针或逆时针数据帧上某个子节点发送数据给下游子节点（节点之间数据互传，不需要主节点参与）。

4) 在顺时针或逆时针数据帧上某个子节点发送数据给上游子节点（节点之间数据互传，需要数据帧环回到主节点再转发）。

对于情况 1、2 和 4， 因为数据帧都会环回到主节点，所以在主节点的两侧放置监听节点即可监听所有的数据。

对于情况 3，目前的设计存在一定局限性，只在主节点两侧放置监听节点是不能有效监听数据的，需要用户根据实际应用场景在环路中某些重要节点增加监听节点。在环路带宽允许的前提下，情况 3 也可以用情况 4 替代。

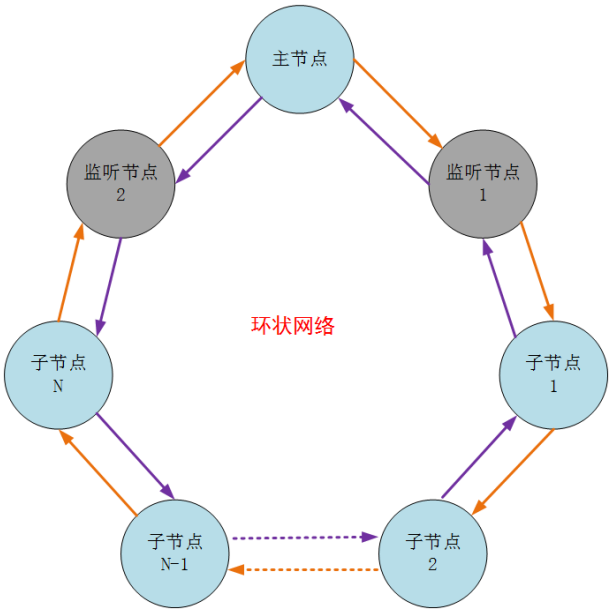


图 9-1 监听节点